

同濟大學

TONGJI UNIVERSITY

毕业设计（论文）

课题名称 实时协同编程环境中的共享终端技术与原型系统实现

副标题 _____

学 院 软件学院

专 业 软件工程

学生姓名 方必诚

学 号 2052329

指导教师 范鸿飞

日 期 2024-06-01

实时协同编程环境中的共享终端技术与原型系统实现

摘要

实时协同编程是一种新兴的协作开发模式，允许多位程序员同时编辑共享的源代码，每位程序员的编辑操作会被即时自动地传送给其他协作成员。这种方式极大地提升了开发的实时性和效率。共享终端作为其中的一种协作功能，使得地处不同地点的团队成員可以实时共同观看和控制同一命令行界面，从而在实时环境下进行协作。这一功能在教育、技术支持和软件开发等多个领域均发挥了一定的作用。

然而，现有的共享终端存在以下两个主要问题：首先，只有主持人能够发起共享终端会话，其他参与者无权启动会话，这限制了共享终端的灵活性和便捷性；其次，现有的权限设置较为单一，通常只提供完全只读和完全读写两种模式。完全只读模式适用于教育演示等单向信息传递的场景，但在需要高度协作的工作环境中显得不够灵活。完全读写模式虽然赋予了所有参与者高度的操作自由，但也带来了安全性风险，可能导致宿主机面临数据安全和隐私泄露的威胁。

为了解决上述问题，本文设计并实现了一种新型的无约束的多层次权限安全共享终端方案（FlexSecure），旨在提高协作的灵活性和安全性。在该方案下，每个参与者都被赋予了启动新的共享终端会话的权限，而不仅仅是会议主持人掌控会话的启动权。为了增强权限控制的灵活性，除了现有的完全只读和完全读写模式外，我们引入了更精细的权限控制功能。该功能根据预设规则，智能地排除特定指令的执行权限，对存在风险的指令发起警告或禁止，在不干扰正常协作的前提下，有效保护发起终端者的系统安全和个人隐私。

本文基于 CoVsCode 实时协作平台，初步实现了 FlexSecure 共享终端方案的原型系统，并对其进行了功能和性能方面的测试，验证了各类方法与技术的可行性。我们还邀请了参与者对该系统进行可用性评估，并根据反馈进一步优化了系统设计。

关键词：实时协同编程，共享终端，FlexSecure

Shared Terminal Technology And Prototype System Implementation In Real-time Collaborative Programming Environment

ABSTRACT

Real-time collaborative programming is an emerging development method that allows multiple programmers to simultaneously edit shared source code, with each programmer's edits being instantly and automatically transmitted to other collaborators. This method significantly enhances the immediacy and efficiency of development. The shared terminal, as a collaborative feature, enables team members located in different places to jointly view and control the same command line interface in real-time, facilitating collaboration in a live environment. This feature plays a role in many fields such as education, technical support, and software development.

However, existing shared terminals have two main issues: first, only the host can initiate a shared terminal session, while other participants have no right to start the session, limiting the flexibility and convenience of the shared terminal. Second, the current permission settings are relatively simplistic, usually offering only fully read-only and fully read-write modes. The view-only mode is suitable for scenarios involving one-way information transmission, such as educational demonstrations, but it lacks flexibility in highly collaborative work environments. Although the full access mode grants all participants a high degree of operational freedom, it also introduces security risks, potentially exposing the host machine to data security and privacy threats.

To address these issues, this paper designs and implements a novel unconstrained multi-level permission security shared terminal solution (FlexSecure), aiming to improve the flexibility and security of collaboration. In this solution, each participant is granted the permission to initiate new shared terminal sessions, not just the host controlling the session initiation right. To enhance the flexibility of permission control, in addition to the existing fully read-only and fully read-write modes, we introduce more refined permission control features. This feature intelligently excludes the execution permission of specific commands according to preset rules, issues warnings or prohibits risky commands, effectively protecting the system security and personal privacy of terminal initiators without disrupting normal collaboration.

Based on the CoVsCode real-time collaboration platform, this paper preliminarily implements a prototype system of the FlexSecure shared terminal solution and conducts functional and performance testing, verifying the feasibility of various methods and technologies. We also invited participants to evaluate the usability of the system and further optimized the system design based on feedback.

Key words: real-time collaborative programming, shared terminal, FlexSecure

目 录

1 引 言.....	1
1.1 课题背景.....	1
1.2 两种协同编程方式.....	1
1.2.1 非实时协同编程.....	2
1.2.2 实时协同编程.....	4
1.3 共享终端.....	6
1.3.1 基本介绍.....	6
1.3.2 共享终端的潜在问题.....	8
1.4 研究内容.....	9
1.5 本文结构.....	9
2 技术背景.....	11
2.1 现有实时协作编程与共享终端方案.....	11
2.1.1 Visual Studio Code Live Share	12
2.1.2 JetBrains Code With Me	13
2.1.3 Upterm	14
2.2 插件开发环境.....	17
2.2.1 Visual Studio Code.....	17
2.2.2 CoVsCode	19
2.2.3 GoLand.....	20
3 实时协同编程环境中的共享终端技术.....	22
3.1 现有共享终端方案的不足.....	22
3.2 无约束的多层次权限安全共享终端方案（FlexSecure）	25
3.3 FlexSecure 方案流程.....	26
3.3.1 更加灵活的共享终端.....	26
3.3.2 更加安全的协作方式.....	27
4 系统设计与实现.....	29
4.1 需求分析.....	29
4.1.1 功能性需求.....	29
4.1.2 非功能性需求.....	30
4.2 关键设计与实现.....	30
4.2.1 在 CoVsCode 中实现共享终端	30
4.2.2 共享终端中的细化权限控制.....	32
5 系统运行与测试.....	35
5.1 插件的安装与运行.....	35
5.1.1 插件安装.....	35
5.1.2 插件运行.....	37
5.2 综合测试.....	44
6 总结与展望.....	46
6.1 总结.....	46
6.2 展望.....	46
参考文献.....	48
谢 辞.....	51

1 引言

在本章中，我们将对项目的背景和研究目标进行概述，并简要介绍实时协同编程和共享终端技术的相关概念和现状。通过深入探讨该领域的发展趋势和所面临的挑战，我们将展现本文所提方案的研究价值和现实意义。同时，我们还将详细阐述本文所取得的工作成果，包括但不限于研究成果、创新点以及实际应用前景，为读者提供初步认知。最后，我们将对论文的整体结构进行概括说明，帮助读者更好地理解和把握论文的核心内容和逻辑框架。

1.1 课题背景

在当今时代，随着计算机软件工程的迅猛发展，软件已经渗透到我们的生活和工作的方方面面。金融、医疗等各行各业对于复杂大型软件的需求不断增长，这促使软件系统不断朝着大规模、高复杂性的方向发展。然而，这种发展趋势也带来了开发成本的急剧增加、团队规模的扩大以及系统结构的日益复杂化的挑战。为了有效地解决这些挑战，缩短开发周期，提升项目质量，协同编程这一方法得以崭露头角。协同编程通过模块化的开发和分工协作，极大地提高了开发效率和软件质量，为整个软件工程领域带来了革命性的变革。

据研究显示，在软件开发的实际过程中，开发人员大约花费了 70%的时间用于协作和沟通，这说明提高协同合作效率在降低成本方面起到了关键的作用。目前，业界为满足不同的开发需求，广泛采用了非实时和实时两种协同编程方式^[1]。随着项目规模的持续扩大和远程协作需求的不断增长，开发人员之间的协同编程紧密度也日益增强^[2]。为此，工业界和学术界都在协同编程的研究与应用上投入了大量的资源，旨在不断优化和提升协同编程的效率和效果。

在全球经济信息网络的新时代，软件开发行业面临着开发周期缩短、项目规模扩大以及远程协作需求增加等多重挑战^[3]。在这个背景下，开发团队需要灵活应对成员间的编程能力和经验差异，合理安排工作以满足多样化的开发需求。协同编程在软件开发流程中扮演着至关重要的角色，通过模块化分解复杂系统，促进不同技术背景的开发者的分工合作，不仅显著提升了项目的开发速度，还确保了软件的高质量标准。

1.2 两种协同编程方式

在协同编程中，开发人员可共同编辑同一组源代码。每名开发人员的编辑操作都会通过特定技术，同步到其他开发人员的本地站点，确保各站点源代码的一致性。根据编辑操作同步的时间点，协同编程技术主要分为两类：非实时协同编程和实时协同编程。两种协作方式的主要区别如表 1.1 所示。

表 1.1 非实时协同编程与实时协同编程的比较

比较项目	非实时协同编程	实时协同编程
协作方式	开发者各自在本地环境中独立进行开发	多个开发者同时在一个代码文件上进行编辑和修改
协同时间	短，开发者独立完成任务后再进行合并	长，持续的实时互动
冲突解决	可能需要手动合并和解决代码冲突	自动化冲突解决机制，减少人为干预
适用的项目类型	适合任务划分明确，开发者可以独立完成的项目	适合难以明确划分任务的项目
适用的项目规模	适用于大型项目或任务划分明确的大团队项目	适用于小型项目或需要高频互动的小团队项目
交流频率	交流频率较低，通常通过定期会议或代码评审进行	高频实时交流，方便即时沟通
代码质量	由于缺少实时反馈，代码审查和改进的机会相对较少	实时审查和改进，提高代码质量

1.2.1 非实时协同编程

非实时协同编程（也被称为异步协同编程），是一种相对宽松、对实时性要求不高的协同开发方式。它允许多名程序员在相同的项目代码中并行工作，独立地查看和修改代码，从而实现各自编程任务的高效完成。非实时协同编程主要依赖版本控制系统来保持代码同步。每位程序员在本地环境中独立进行开发工作，互不干扰。在适当的时候，他们将本地修改的代码提交到远程服务器进行合并与同步。

这种协同编程方式的交流频率相对较低，程序员之间的协同工作具有较高的并行度。然而，这也带来了一定的挑战。由于开发人员在代码同步前无法实时看到其他人的代码变更，因此代码冲突的可能性增加。一旦冲突发生，开发人员需要手动解决这些合并冲突，以确保代码的一致性和项目的顺利进行。尽管如此，非实时协同编程仍然是一种非常实用的开发方式，特别适用于对实时性要求不高，但需要高效并行开发和灵活团队协作的项目。

在非实时协同编程中，开发人员会共享一个文件工程，其中包含了整个项目的目录结构以及所有的源代码文件。这种协同模式的特点是编程模块之间的耦合度相对较低，使得开发人员能够在相对独立的环境中工作，减少了相互之间的干扰。然而，这也意味着协作的紧密程度相对较低，需要开发人员更加自觉地维护代码的一致性和规范性。

在这种模式下，开发者需要将远端的代码仓库拉取到本地，并在本地进行修改。一旦修改完成，他们会尝试将变更推送到远程仓库进行合并。这一过程并不是实时的，只有当开发人员主动请求并获取远端的最新更新时，他们才能了解到其他团队成员的最新代码变更。这种异步协作方式虽然为开发者提供了更加灵活的工作方式，但也可能在一定程度上降低开发的效率。

版本控制系统在非实时协同编程中发挥着至关重要的作用^[4]。主流的版本控制系统架构包括集中式和分布式两种。集中式版本控制系统（Centralized Version Control System, CVCS），如 CVS 和 Subversion（SVN）^[5]，以中央服务器为核心，开发人员通过客户端连接服务器来获取或提交代

码。其示意图如图 1.1 所示。这种架构的优点是便于管理和维护，但缺点是网络依赖性强，且容错能力相对较弱。一旦中央服务器出现问题，将可能影响整个开发流程和项目数据的安全。

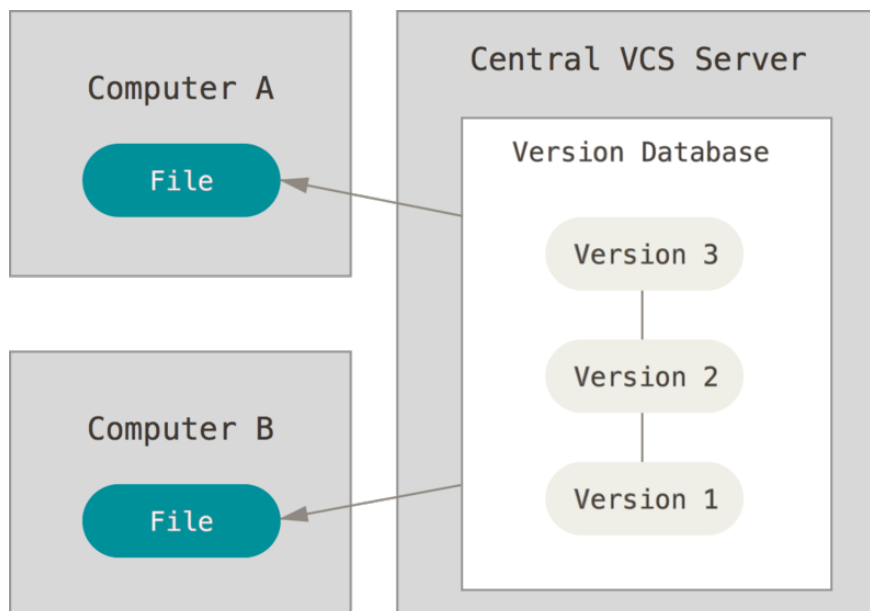


图 1.1 集中式版本控制系统的示意图

而分布式版本控制系统，如 Git 和 Mercurial，则去中心化了版本库管理。每个开发者都拥有代码仓库的完整副本，可以在本地进行提取和提交操作，这些操作都会形成仓库的完整备份。这种架构减少了单点故障的风险，提高了数据恢复的可行性。由于每个开发者都有完整的代码库副本，即使中央服务器出现问题，开发者仍然可以在本地进行开发工作，并在服务器恢复后同步代码。每个本地仓库都是完整备份，一旦某个开发者的仓库损坏，可以从其他开发者的仓库中恢复数据。目前，Git^[6]是应用最广泛的分布式版本控制系统之一。如图 1.2 所示，通过不同的分支和提交记录管理，Git 可以实现较好的历史版本管理。

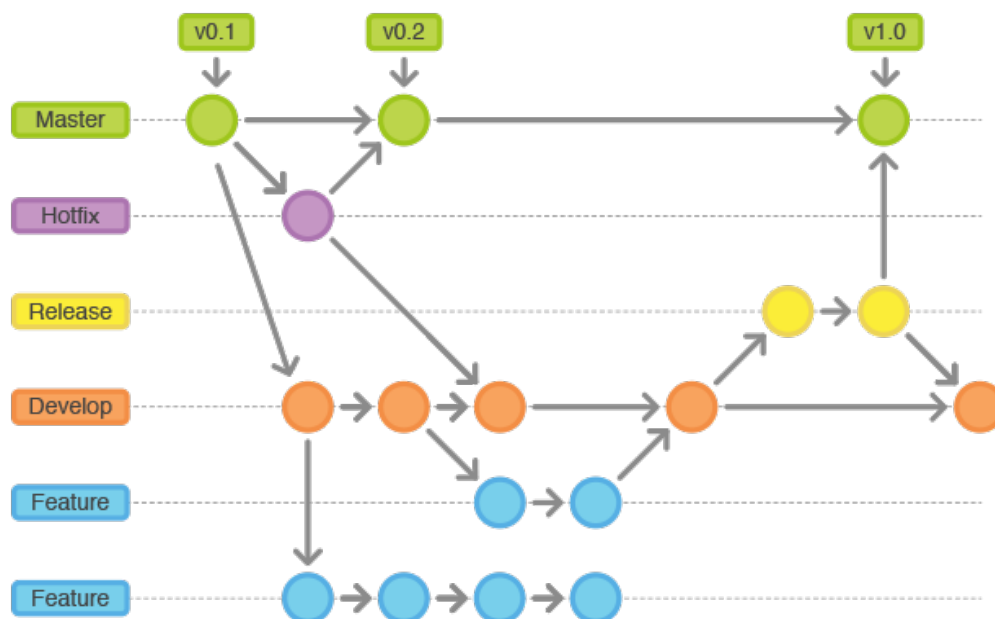


图 1.2 Git 的使用示意图

尽管非实时协同编程通过版本控制系统维护了代码的一致性，降低了协同工作的复杂性，但在高耦合度的编程模块中，开发人员仍然可能面临频繁的代码合并冲突。手动解决这些冲突不仅增加了协作的学习成本，也可能降低开发效率。此外，分布式版本控制系统的操作和概念较复杂，开发者需要一定的学习成本。尽管如此，非实时协同编程仍然是一种成熟的协同编程模式，广泛应用于工业界，为团队协作和项目管理提供了有效的解决方案。

1.2.2 实时协同编程

实时协同编程是一种新兴的技术，它极大地推动了协作开发模式的进步，使多名开发者能够同时、无缝地在相同的源代码上进行编辑。这一模式的优势在于，任何协作者在界面上所做的任何更改，都会立即同步到其他所有协作者的工作环境中，而无需任何的手动干预。底层系统会自动处理所有合并和同步的复杂过程，从而彻底消除了手动解决冲突的需求。在涉及高耦合度的源代码的复杂场景中，实时协同编程允许开发者专注于代码的核心功能，极大地提升了团队的整体开发效率。

实时协同编程以其实时性、直观性和自动化特性，弥补了许多传统非实时协同编程的缺陷，在工业和教育领域具有巨大的发展潜力^[7]。在大型软件系统的开发中，实时协同编程能够帮助团队成员即时地解决模块间的依赖问题，确保代码的一致性和稳定性^[8]。在结对编程中，即使程序员们分布在世界的各个角落，也能通过互联网进行实时、高效的协作^[9]。而在计算机专业的教学中，实时协同编程使得老师和学生可以共同编辑代码，实时讨论和解决问题，从而极大地提高了教学和学习效果^[10]。

目前市场上已经涌现出多款实时协同编程产品，如微软的 Visual Studio Live Share、GitHub 的 Teletype for Atom、JetBrains 的 Code With Me、Genuitec 的 CodeTogether 以及 Replit 等。Visual

Studio Live Share^[11]是微软推出的一款实时协同开发工具，允许开发者在 Visual Studio 和 Visual Studio Code 中共享代码、调试会话和终端，并支持跨平台协作。Teletype for Atom^[12]是 GitHub 为 Atom 编辑器开发的插件，通过 WebRTC 技术实现实时协同编辑，允许多名用户同时编辑同一文件，并且可以实时看到彼此的光标位置和编辑操作。Code With Me^[13]是 JetBrains 开发的一款协同工具，集成在其 IDE 中，提供安全的实时协作环境，支持共同编辑代码、调试和运行项目，适用于团队开发和远程教育。CodeTogether 是 Genuitec 推出的一款支持多种 IDE 的协同编程工具，通过浏览器或 IDE 插件实现实时代码共享和协作编辑，具有灵活的权限控制和安全加密功能。Replit 则是一款基于云的编程平台，提供在线编写、运行和共享代码的功能，支持多种编程语言，方便用户随时随地进行协作开发。Coder^[14]是一款完全基于云端的开发环境，允许开发者通过浏览器进行实时协作编程，无需本地安装任何开发工具。这些工具各具特色，满足了不同用户在协同编程方面的需求。在协作模式方面，这些工具都采用了主机协作者模式，允许一名开发者作为主机分享项目，并邀请其他协作者加入进行实时编辑。尽管这些工具极大地提升了协作效率，但它们也存在一些限制，例如主机通常拥有更高的权限，协作者在主退出后可能无法继续编辑等。在图 1.3 中，两名程序员在同一个项目中进行协作，双方都能实时看到彼此的工作情况，这有利于促进协作的效率。

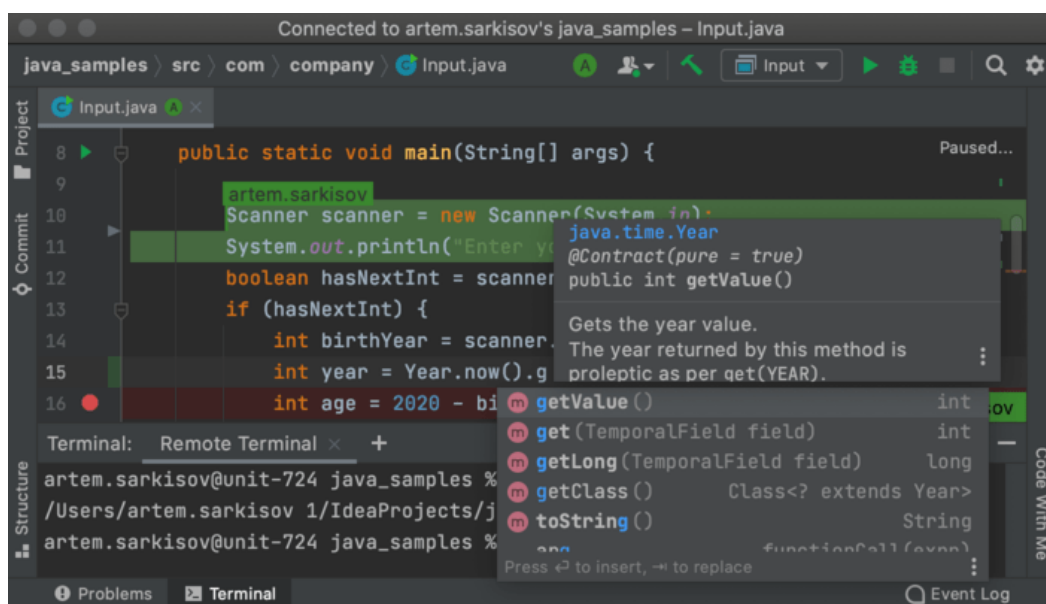


图 1.3 Code With Me 的使用示意图

与非实时协同编程相比，实时协同编程支持更加紧密、高效的合作方式。它通过实时更新和自动化合并改动，有效地避免了手动解决冲突的繁琐过程，为团队提供了一个高度互动的工作环境。在这样的环境中，开发者可以即时讨论问题，分享见解，从而减少了沟通成本，避免了由于信息滞后导致的重复工作或错误修正^[15]。对于企业来说，这意味着可以更快地交付高质量的软件产品，保持市场竞争力。

此外，实时协同编程对开源项目也具有重要的推动作用。在开源社区，开发者来自全球各地，时区和工作时间的差异使得协作变得复杂。实时协同编程工具的引入，打破了这一障碍，使得全球开发者可以更加无缝地协同工作，加速项目的进展。

尽管实时协同编程工具为团队合作和远程协作带来了许多便利，但也存在一些局限性和缺点^[16]。首先，网络连接稳定性是实时协同编程的关键因素之一。如果团队成员之间的网络连接不稳定或者延迟较高，可能会导致协同编辑的实时性受到影响，甚至出现同步错误或数据丢失的情况^[17]。其次，安全性是一个值得关注的问题。在实时协同编辑过程中，可能涉及到敏感信息或者私密数据，如果协同工具的安全性无法得到保障，可能会存在信息泄露或者非法访问的风险。此外，对于大型项目或者复杂的代码结构，实时协同编辑工具的性能可能会受到限制，导致编辑、编译和调试过程变得缓慢或者不稳定。最后，不同的团队成员可能习惯于使用不同的开发环境和编辑器，而某些实时协同编程工具可能并不完全支持所有的开发环境，这可能会导致一些兼容性和一致性的问题。因此，在选择和使用实时协同编程工具时，需要综合考虑其性能、安全性、稳定性以及与团队成员的兼容性等因素。

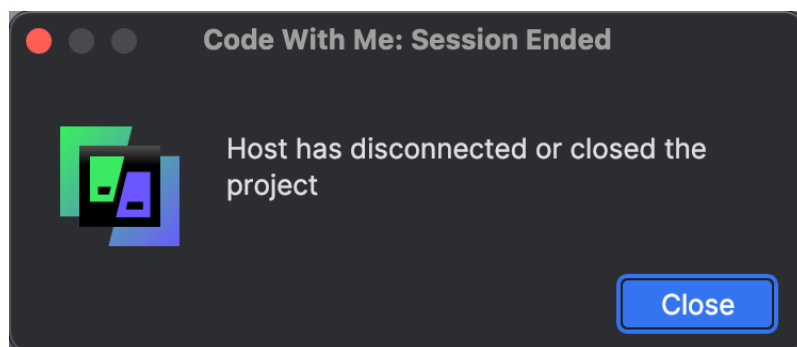


图 1.4 由于网络原因，团队成员与项目之间的连接被断开

实时协同编程的工作流程通常包括创建代码仓库、发起实时协同工作区连接请求、加入工作区和在共享工作区内进行协作编辑。这种方式的分布式特性意味着，只要有网络连接，开发者们就可以随时参与到实时协作中。无论是大型模块的共同开发，还是敏捷开发的快速迭代，实时协同编程都能提供强大的支持，展现出其广泛的应用范围和巨大的发展潜力。

1.3 共享终端

本节将对实时协同编程中的共享终端功能进行简单的介绍，并对其潜在问题做初步的说明。

1.3.1 基本介绍

共享终端技术的出现，为远程团队的协作带来了新的改变，使得身处不同地点的团队成员，能够实时地共同观看和控制同一命令行界面，从而在实时的环境下协同工作。在现代软件开发领域，随着远程工作和敏捷开发模式的普及，共享终端技术正扮演着越来越重要的角色。

共享终端的特点在于，它提供了一个高度同步的视图和交互环境。无论是输入指令还是查看输出结果，所有的操作都会在参与者之间实时共享，确保了团队协作的透明度和流畅性。这种实

时的互动和沟通方式，大大减少了信息传递的延误和误解，提升了团队的整体协作效率。

在教育领域，共享终端的应用为技术教学带来了很大的便利。教师可以通过共享终端，实时演示命令行操作技巧，而学生则可以即时跟随操作，进行实践练习。这种互动的教学方式，不仅提升了学生的学习兴趣，更使得技术教学的质量和效率得到了提升。

在技术支持方面，共享终端同样展现出了巨大的潜力。通过共享终端，技术专家可以远程查看和操作用户的终端，从而为用户提供及时有效的技术支持。这种即时的支持方式，不仅提高了问题解决的速度，更使得技术支持变得更加高效和便捷。

在软件开发领域，共享终端可以成为团队协作的有效工具。开发人员可以通过共享终端，共同查看运行结果，实时交流思路和策略，从而更加高效地完成开发任务。同时，共享终端还能够增强团队成员之间的粘性，促进团队成员之间的交流和合作。

值得一提的是，共享终端与实时协同编程有着密切的联系。实时协同编程不仅包含了共享终端的所有功能，还进一步扩展到了代码编写、查看、运行和调试等多个方面。而共享终端则更加专注于命令行会话的共享。当这两者相结合时，团队成员不仅可以实时观察代码的变更，还能够共同体验代码运行过程中的所有细节，从而进一步提升团队协作的质量和深度。

如今，越来越多的开发工具开始集成共享终端功能。例如，Visual Studio Code 的 Live Share 扩展不仅支持代码共享，还允许用户共享终端。这使得远程团队成员能够共同执行命令、启动服务器或调试程序，极大地增强了团队的协作能力。而 JetBrains 的 Code With Me 功能也提供了类似的体验，使得使用其 IDE 的用户能够轻松开始共享终端会话。这些工具的出现，不仅使得代码编写的协作变得更加简单和直观，也使得命令行任务的协作变得更加容易。

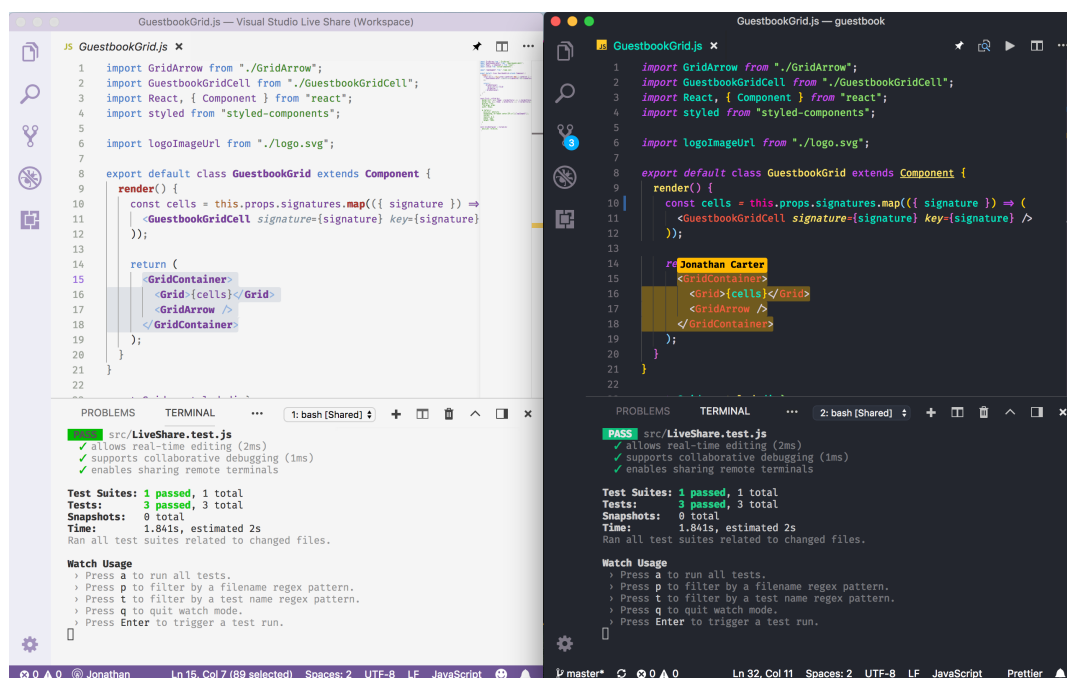


图 1.5 Visual Studio Code Live Share 的共享终端功能

如图 1.5 所示，两位程序员通过在 Visual Studio Code 中使用 Live Share 插件，共同开发同一个项目。除了代码编辑区，他们还可以在同一个终端中实时协作，这在一些特殊的情况下特别有用。例如，当某位程序员需要在本地测试开发的代码，但尚未配置好本地环境时，他可以通过其他已经配置好环境的程序员的终端来测试代码，从而节省时间并提升效率。在 CI/CD（持续集成和持续部署）流水线中，多个团队成员可以通过共享终端实时监控构建和部署的过程，快速响应并解决出现的问题。这种实时的协作方式，能够有效减少部署失败的风险，提高软件交付的速度和可靠性。

由此看来，共享终端在现代软件工程实践中扮演着重要的角色。它通过提供实时、交互式和易于访问的命令行共享环境，极大地促进了教育、远程支持和团队协作。随着远程工作和敏捷开发方法的不断普及，我们有理由相信共享终端技术将继续在软件开发和计算机教育领域发挥重要作用。

1.3.2 共享终端的潜在问题

由上文可知，共享终端技术作为一种重要的创新，为多人协同编程、资源共享以及信息交流提供了坚实的平台支撑。它极大地提升了工作效率，使多地点同步作业成为可能，为团队协作注入了新的活力。然而，经过笔者调研与分析发现，尽管共享终端技术在实际应用中展现出许多优越性，但其现有的实现方式仍然存在着不容忽视的问题与局限性，这些问题在一定程度上对共享终端的发展潜力和应用广度构成了制约。

首先，目前市面上广泛应用的共享终端产品大多存在一个明显的限制，即只有宿主机（Host）能够发起共享终端会话，而其他参与者则无权启动该会话。这种设计极大地限制了共享终端的灵活性和便捷性，当宿主机无法及时发起共享时，整个协作进程可能会受到阻碍，甚至导致协作无法顺利进行。例如，在一个跨时区的团队中，如果宿主机的拥有者因时差或其他原因无法及时在线，其他成员便无法继续进行工作，从而严重影响了整体的工作效率。

其次，现有的共享终端在权限设置上普遍较为单一，大多只提供完全只读和完全读写两种权限模式。完全只读权限模式虽然适用于教育演示等单向信息传递的场景，但在需要高度协作的工作环境中则显得捉襟见肘。在该模式下，其他参与者无法进行实时的互动和修改，这无疑降低了协作效率，使得团队协作变得困难重重。例如，在一个软件开发项目中，团队成员需要频繁地进行代码修改和调试工作，如果只能通过宿主机进行操作，其他成员无法实时参与修改和调试，必然会拖慢项目进度。

此外，完全读写权限模式虽然赋予了所有参与者高度的操作自由，使得协作互动变得无障碍，但同时也带来了安全性威胁。在这种模式下，所有参与者都有可能访问和修改宿主机电脑中的任意文件夹，这不仅使得宿主机的敏感信息暴露于协作成员面前，还可能导致其面临潜在的数据安全和隐私泄露风险。例如，在一些涉及商业机密或敏感数据的项目中，完全读写权限模式下的共享终端可能会被恶意利用，导致数据泄露和信息安全事故的发生。

虽然共享终端技术为现代工作方式带来了革命性的创新和便利，但现阶段其产品实现在权限管理、操作灵活性以及安全性等方面仍存在诸多不足。这些问题的存在不仅限制了共享终端的应用范围，也为用户的数据安全和隐私保护带来了隐患。因此，未来的研究和产品开发必须针对这

些问题提出切实可行的改进方案，以推动共享终端技术的广泛应用和健康发展。同时，用户在使用共享终端时也应保持警惕，确保个人数据的安全和隐私的保护。

1.4 研究内容

本文通过对现有共享终端工具的深入剖析与改进，提出了一种新型的无约束的多层次权限安全共享终端，以提升其灵活性与安全性，从而更好地服务于实时协同编程的工作场景。具体研究内容涵盖以下几个方面：

1. 深入探究现有共享终端的工作原理与机制

这一环节要求对现行共享终端的工作原理和机制进行全面而系统的分析，包括但不限于其通讯协议、数据同步算法、权限控制策略以及用户界面设计。通过深入剖析这些原理和机制，我们能够明确现有系统的优势所在，同时发现其不足之处，为后续的研究工作提供理论基础和设计改进的出发点。

2. 设计新型共享终端机制及优化操作流程

在前期研究的基础上，我们将设计一种新型的无约束的多层次权限安全共享终端机制，并优化其操作流程。新机制将特别注重在保障数据安全的同时，提供更高的操作灵活性。针对实时协同编程的特殊需求，我们将设计合理的权限分配模式、会话管理框架以及实时交互的数据处理流程，以构建一个适应性更强、安全可靠的协同编程环境。

3. 实现共享终端原型系统并进行功能验证

我们选定 Visual Studio Code 作为实现平台，借助其跨平台特性和丰富的插件支持体系，基于新设计的共享终端机制开发原型系统。在这一阶段，我们将重点关注实现方案的可行性验证，包括但不限于共享终端的核心功能开发、用户界面设计、以及后端数据处理等关键技术的具体实现。我们将通过一系列的功能测试和性能评估，验证原型系统的稳定性和准确性。

4. 开展综合用户测试并优化系统设计与功能

原型系统开发完成后，为了全面评估系统性能和用户体验，我们将开展一系列综合的用户测试。这些测试不仅涵盖了功能性测试，以确保系统的稳定性和准确性，还包括了性能测试和用户体验调研，以评估系统在实际应用环境中的表现。我们将认真倾听用户反馈，并根据反馈结果进一步优化系统设计和功能实现，以提升用户体验和满足更多实际需求。

综上所述，本研究致力于推动共享终端技术的发展，通过设计与实现新型共享终端机制，旨在满足实时协同编程环境下的特定需求，以促进团队合作的高效与安全。我们期待通过这一研究，为实时协同编程领域带来更加便捷、安全和高效的共享终端解决方案。

1.5 本文结构

第一章为引言，详细阐述了本研究的背景与动机。首先，对协同编程的概念进行了介绍，并概括了非实时协同编程与实时协同编程两种模式的特点。接着，探讨了共享终端的运作原理、其优势以及目前的局限性，为本研究的内容与论文的整体结构提供了清晰的概述。

第二章为技术背景，深入探讨了插件开发环境、插件的文件系统架构以及当前共享终端的解决方案等相关技术。这一章为后续章节的深入研究提供了技术基础。

第三章聚焦于实时协同编程环境中的共享终端技术。通过对现有技术的不足进行深入分析，提出了具有新型的无约束的多层次权限安全共享终端优化方案，并对其主要流程进行介绍。

第四章为系统设计与实现。首先进行了详细的需求分析，包括功能性需求和非功能性需求。随后，重点介绍了系统的关键设计与实现过程，确保了系统的有效性和稳定性。

第五章描述了系统的运行与测试。详细阐述了插件的安装与运行流程，并通过具体的使用案例展示了系统的操作流程和实际效果。

第六章为总结与展望。总结了本研究的主要成果和贡献，并对未来的研究方向和发展趋势提出了展望，为相关领域的研究提供了有价值的参考。

2 技术背景

2.1 现有实时协作编程与共享终端方案

在当前的软件开发领域中，实时协同编程已经逐渐成为一种不可忽视的趋势。这种编程模式不仅极大地提高了团队的协作效率，而且还显著提升了代码的质量。通过实时协同编程，团队成员可以即时共享代码、讨论问题并进行修改，从而避免了传统开发方式中因沟通不畅而导致的误解和重复工作。

在众多支持实时协同编程的工具中，Visual Studio Code Live Share 和 JetBrains Code With Me 是两种不错的共享终端解决方案。

Visual Studio Code Live Share 以其强大的功能集和广泛的兼容性而著称，允许开发者在不同的操作系统和设备上无缝协作，同时还提供了丰富的插件生态系统，以满足各种开发需求。这个工具不仅支持代码共享，还允许用户共享终端，使得远程团队成员能够共同执行命令、启动服务器或调试程序，极大地增强了团队的协作能力。

JetBrains Code With Me 则以其高效的性能和稳定性受到用户的青睐，它提供了同样直观的用户界面和强大的功能，使得团队成员能够轻松地进行代码同步和协作。该工具同样支持共享终端功能，用户可以快速开始共享终端会话。

此外，目前在 GitHub 等开源社区里也有 `tmate`、`gotty`、`webtty`、`Upterm` 等具有一定 star 数量的优秀的共享终端工具。这些工具提供了便捷的共享终端方法，参与者可以通过 SSH 或 Web 页面的方式访问发起者的终端。例如，`tmate` 可以让开发者快速共享一个临时的终端会话，方便其他团队成员进行远程协作和调试。`gotty` 则将终端会话通过 Web 界面共享，使得不论是在本地还是远程的团队都能够轻松访问和使用。

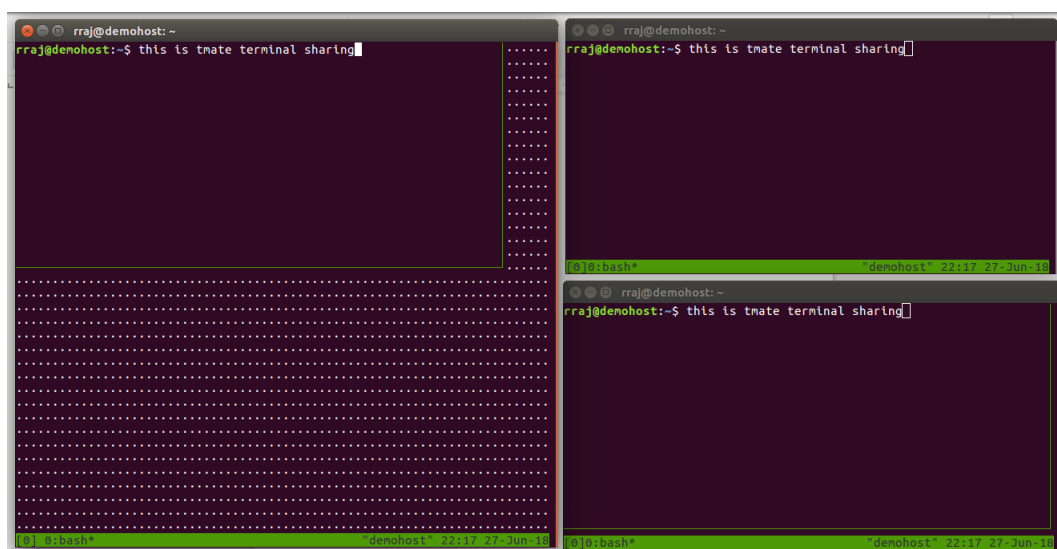


图 2.1 tmate 的使用示意图

在学术界，也有一批实时协同编程的原型工具，这些工具旨在促进开发者之间的协作，如 ATCoPE^[18]，CoIDEA^[19]，CoEclipse^[20]，COLLECE^[21]、CodeSync^[22]、CoMaya^[23]以及基于 Web 的 Collabode^[24]、CoRED^[25]等。这些工具通过提供实时编辑、代码共享和同步等功能，大大提高了开发团队的效率。例如，ATCoPE 和 CoIDEA 在协作编程过程中，通过各种协作机制和工具支持，实现了开发者之间无缝的代码共享和修改，提升了编程的协同性和灵活性。

其中，Ma 等人的研究工作结合了基于 CoEclipse 的实时协作和基于 Git 的非实时协作编程模式，进一步提升了实时协同编程的易用性。这种结合方式不仅保留了实时协同编程的优势，如即时反馈和代码同步，还融入了 Git 系统的版本控制功能，使开发者能够在不同的协作模式之间自由切换，适应各种开发需求和场景。这种混合模式有效地解决了单一协作模式的局限性，提供了更加灵活和高效的协作编程解决方案^[26]。

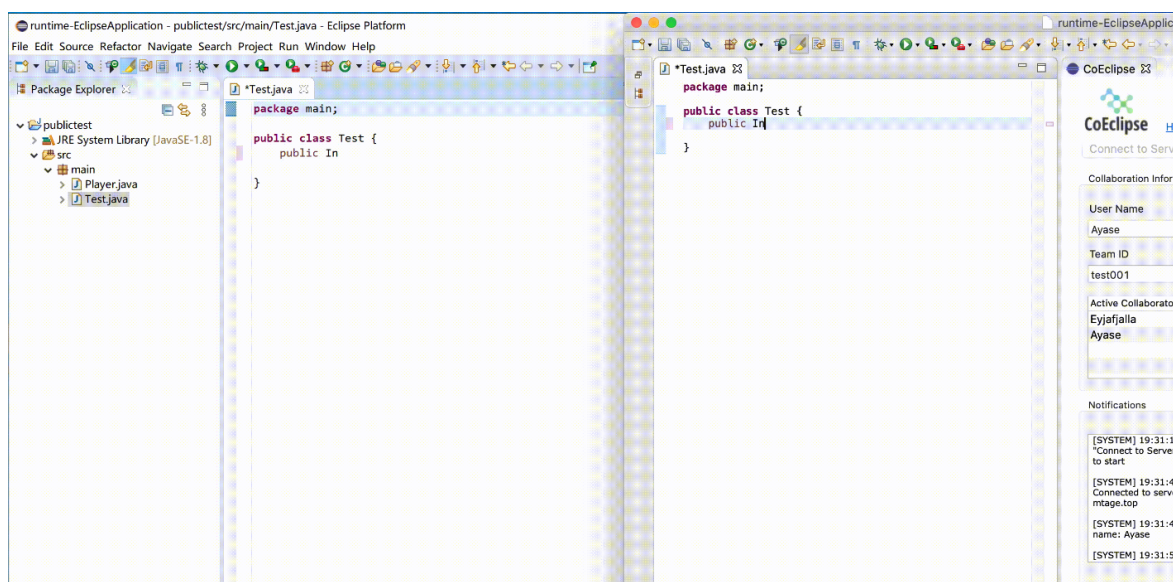


图 2.2 CoEclipse 使用示意图

2.1.1 Visual Studio Code Live Share

Visual Studio Code Live Share 是由微软开发的一项扩展功能，自 2018 年正式推出以来，它已经成为开发者实时协同编辑和调试代码的重要工具。Visual Studio Code 作为一款功能丰富的源代码编辑器，因其对各种编程语言的支持以及全球范围内的广泛应用而备受赞誉。而这款插件的推出，象征着协同编程工具正朝着更加灵活和便捷的方向发展，极大地推动了远程工作和敏捷软件开发的普及与实践。

Visual Studio Code Live Share 具备一系列协同编程的核心功能，包括实时代码共享、同步编辑、断点和调试状态共享，以及共享服务器和终端会话等。这些功能使得团队成员能够跨越地域界限，实现无缝协作，仿佛坐在同一桌旁般高效工作。它有效避免了传统的代码仓库同步问题和版本冲突，为团队成员提供了更加即时和高效的沟通与协作体验^[30]。

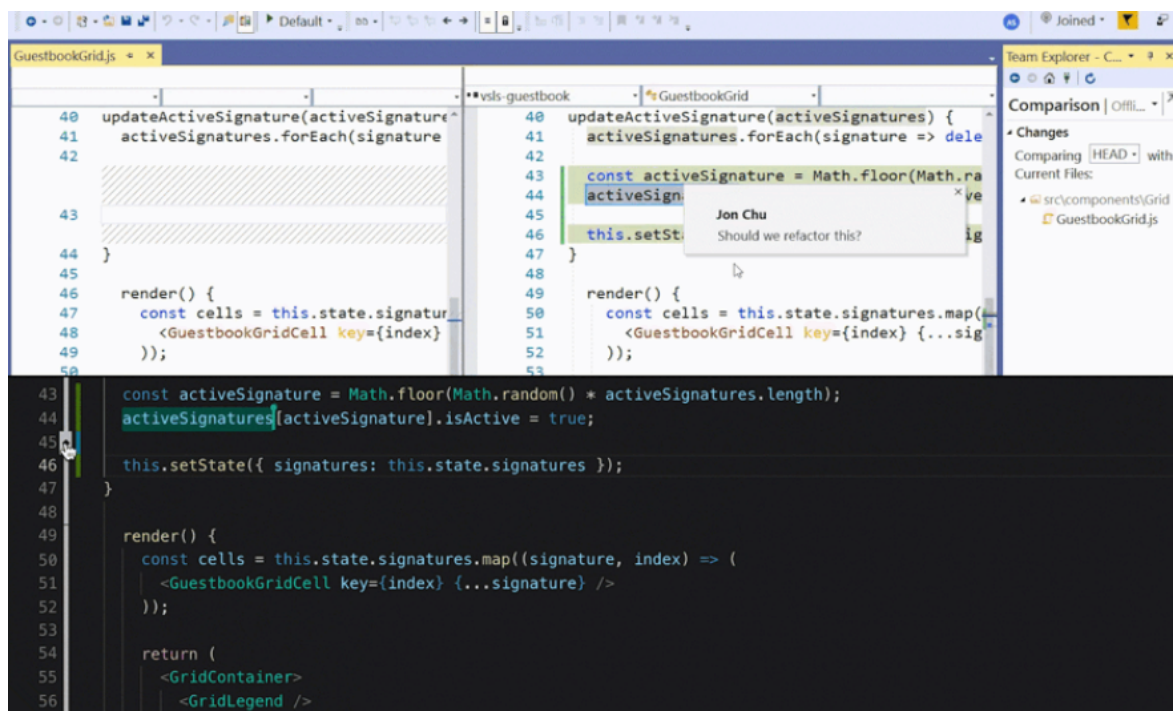


图 2.3 程序员通过 Visual Studio Code Live Share 进行协作和沟通

在本文中，我们将重点关注 Visual Studio Code Live Share 的共享终端功能。该功能允许会话主持人将自己的终端开放给其他协作者，使协作者能够在自己的 Visual Studio Code 环境中轻松访问和使用这个终端。无论是运行脚本、执行命令行构建任务，还是启动服务器，所有参与者都能在实时共享的终端环境中看到相同的输出结果，并可以进行交互式操作。这种无缝集成不仅提高了工作效率，还增强了团队协作的透明度，对于提升团队沟通和解决问题的能力具有重要意义。

2.1.2 JetBrains Code With Me

JetBrains Code With Me 是 JetBrains 公司为其众多集成开发环境(IDE)提供的实时协作工具，该工具自 2020 年推出以来，已经成为开发者进行远程协作的首选解决方案之一^[27]。这项服务的推出彰显了软件开发领域对于提升团队协作效率和编程交互质量需求的响应，同样也是对日益增长的远程工作趋势的一种适应。

Code With Me 的核心竞争力在于其强大的实时协作功能。它允许开发者轻松邀请团队成员通过网络加入自己的编程环境，实现代码的共同查看、编辑和调试。这种跨地域的协作方式极大地促进了团队成员间的即时沟通，使得协同工作更加高效流畅。

在功能层面，Code With Me 提供了丰富的协作特性，包括代码编辑器同步、终端操作共享以及调试过程协作等。这些特性确保了所有参与协作的成员能够在统一的环境中无缝配合，发挥出最大的编程效率。

值得一提的是，Code With Me 与 JetBrains 系列 IDE 的深度集成是其独特优势之一。这种深度集成使得 Code With Me 能够无缝配合 IntelliJ IDEA、PyCharm、WebStorm 等 IDE 的功能，为

用户提供了与本地开发环境几乎一致的远程协作体验。这种体验不仅保证了协作的连贯性和高效性，还大大降低了用户的学习成本。

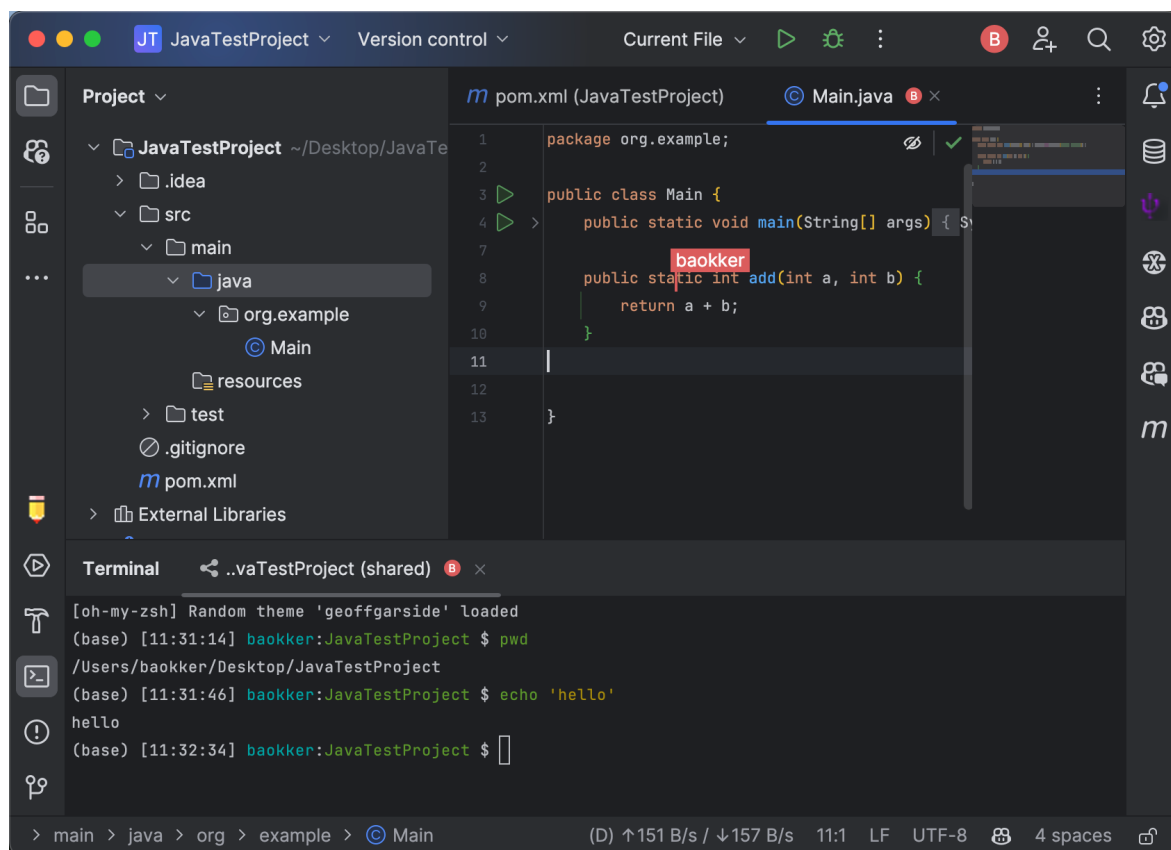


图 2.4 在 Code With Me 中使用共享终端

在本文中，我们将重点关注 Code With Me 的共享终端功能。通过共享终端，协作者们可以实时访问同一个命令行界面，共同执行构建、运行脚本、管理版本控制等任务。这种功能不仅增强了开发流程的透明度，还提高了问题解决的效率。因为所有的调试和环境配置步骤都可以在团队成员之间实时共享和反馈，从而大大缩短了问题解决的时间。如图 2.4 所示，两名协作者者在协作的过程中，除了在代码编辑区域进行协作之外，还可以在共享的终端中同时输入指令。

2.1.3 Upterm

Upterm 是一款由 Go 语言编写的开源终端会话共享工具，能够让开发者通过网络安全地共享终端会话。这款工具非常适用于远程结对编程、访问位于 NAT/防火墙后的计算机、远程调试等场景。其示意图如图 2.7 所示。

这款工具的工作原理如图 2.5 所示：用户首先在主机端启动一个终端会话，并生成一个独特的连接字符串（支持 SSH 或 WebSocket 协议）。随后，其他用户或设备只需利用这个连接字符串，便能轻松接入主机的终端会话。这种方式简化了传统的远程连接流程，不仅提升了使用的灵

活性，也加强了数据传输的安全性。

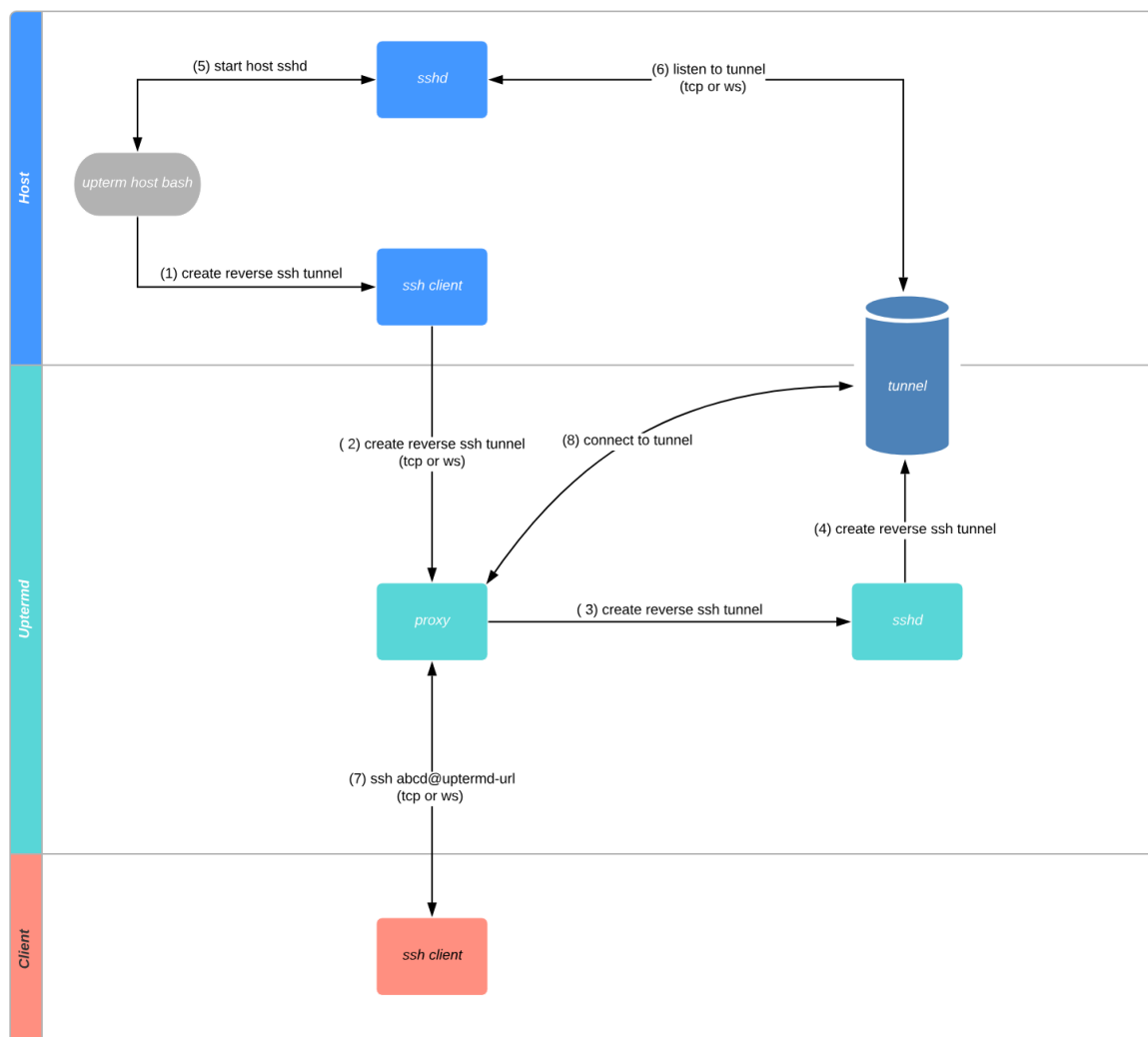


图 2.5 Upterm 结构示意图

安装 Upterm 的方法较为方便快捷。用户可以通过 Homebrew 一键安装（如图 2.6 所示），也可以直接从 GitHub 下载最新的编译好的二进制文件进行安装。Upterm 不仅提供了基本的会话共享功能，还为用户提供了多种高级功能选择。例如，用户可以通过指定客户端的公钥来授权连接，或者将 Upterm 部署到 Kubernetes 集群中，以适应多设备间的远程协作开发环境。



图 2.7 Upterm 使用示意图



总结来看，Upterm 凭借其强大的多功能性和高度可定制性，成为远程协作和终端管理的理想选择。其优秀的性能和广泛的应用场景，使得它值得进行更深入的介绍和研究。

2.2 插件开发环境

2.2.1 Visual Studio Code

本文的技术目标在于将设计的方案实现为原型系统，构建一个基于 Visual Studio Code（简称为 VS Code）插件系统的共享终端工具。此工具旨在利用 VS Code 的出色扩展性，为用户提供实时、高效的终端会话共享功能。为了深入理解并成功实现这一功能，我们需要对 VS Code 的插件开发环境进行一定的介绍。如图 2.8 所示，VS Code 有着极为丰富的插件和广泛的支持。

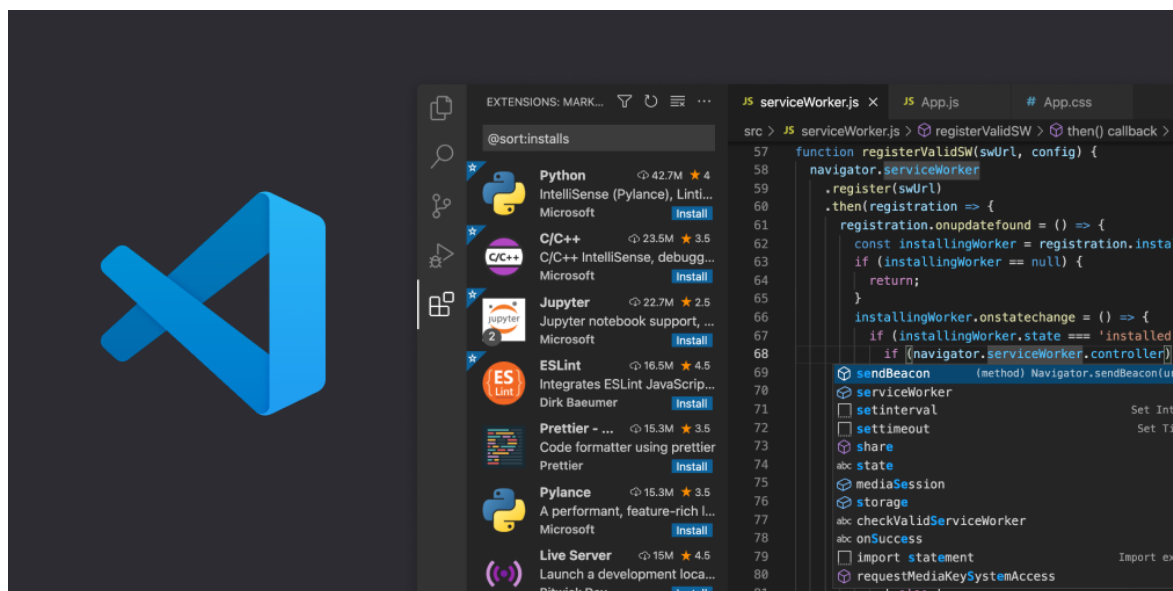


图 2.8 VS Code 及其丰富的插件市场

Visual Studio Code 是一个由微软开发的免费、开源的先进代码编辑器。它支持多种编程语言的开发工作，并且提供了一个强大的扩展框架，允许开发者通过编写插件（Extensions）来扩展编辑器的功能。这些扩展可以通过 VS Code 的扩展市场进行分发和安装。

VS Code 的插件开发环境主要由以下几个核心组件构成：

首先是扩展宿主，VS Code 使用基于 Node.js 的扩展宿主，该宿主运行在与主编辑器进程分离的环境中，负责管理所有插件的生命周期，包括插件的加载、卸载和更新等，同时为插件提供了一套丰富的 API 接口，供插件开发者调用。

其次是 API 接口，VS Code 为插件开发者提供了一套全面而强大的 API 接口^[28]，这些接口允许开发者访问和操作编辑器的各个部分，如工作空间、编辑器窗口、命令、状态栏等。此外，这套 API 还包括了接入调试器和设置系统等关键功能，使得开发者可以方便地扩展编辑器的功能。

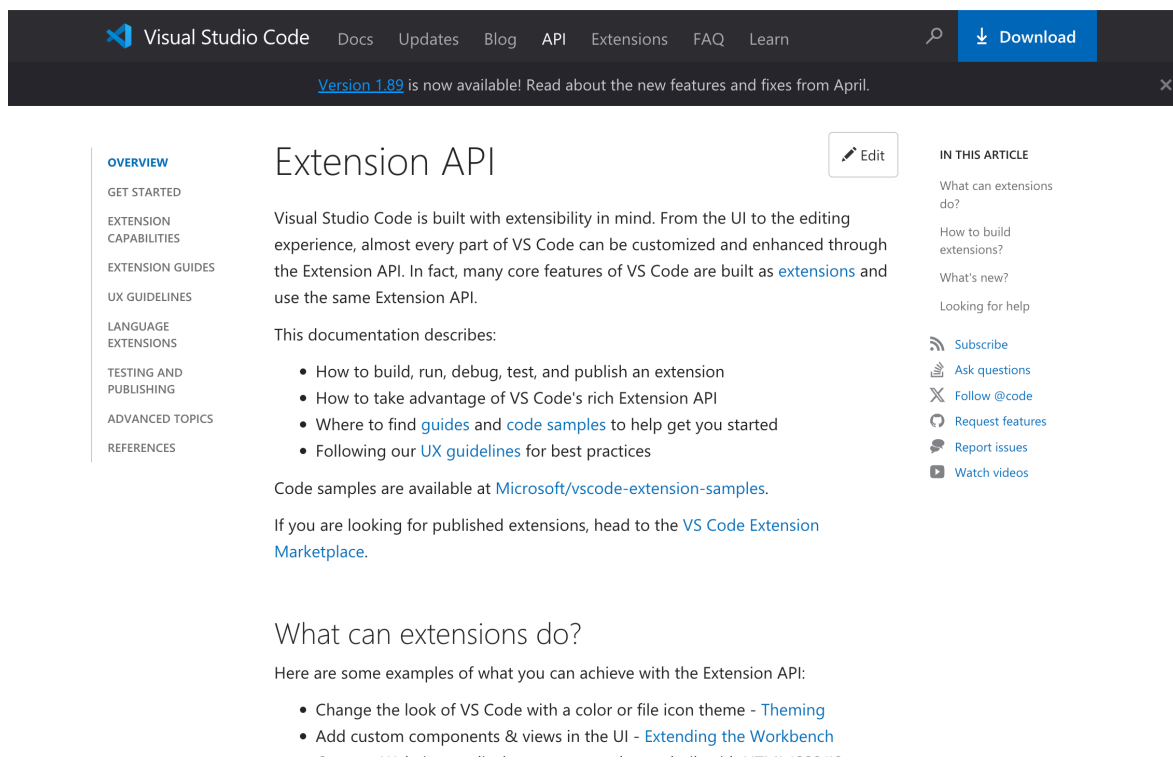


图 2.9 VS Code 的官方插件 API 文档

再者是语言支持，VS Code 插件可以利用内置的语言服务器协议（LSP）来提供对多种编程语言的支持，包括代码自动完成、定义跳转、错误提示等高级功能。这使得 VS Code 能够适应各种编程语言的开发需求，提高开发效率。

此外，VS Code 还提供了调试器 API，使得插件可以集成自定义的调试器，满足特定语言或框架的调试需求。这一功能对于提高插件的可用性和用户体验具有重要意义。

VS Code 插件可以包含自定义的视图和编辑器，这些可视化组件可以嵌入到 VS Code 的活动栏、侧边栏或编辑器区域，为用户提供独特的用户界面和交互体验。这使得插件能够更好地融入 VS Code 的使用环境，提升用户的使用体验。

最后，VS Code 的任务系统（Tasks）和源代码管理（SCM）接口允许插件与外部工具和本地控制系统进行集成，进一步提升了开发效率。这意味着插件可以方便地与其他开发工具和服务进行交互和协作，提高了插件的实用性和灵活性。

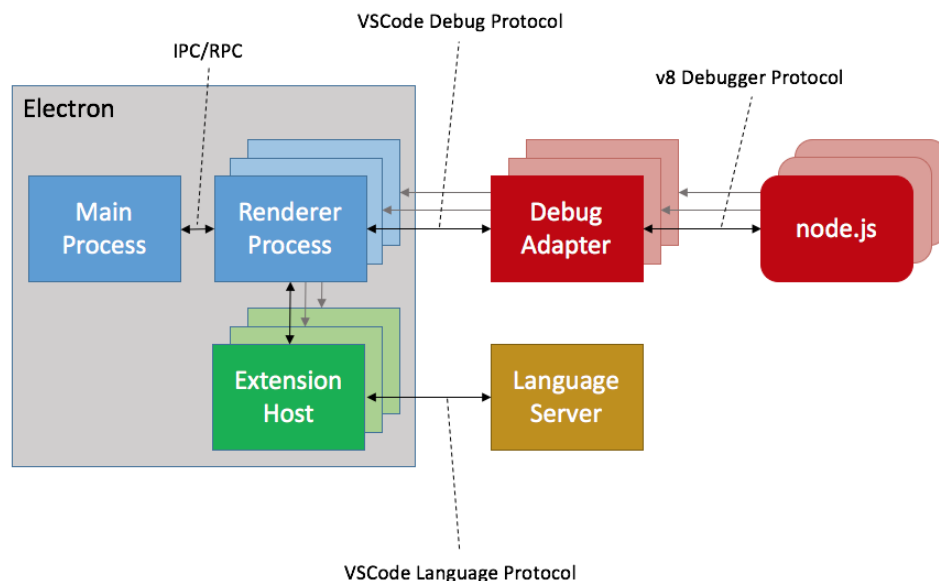


图 2.10 VS Code 的插件开发结构

开发 VS Code 插件的步骤一般如下：

1、环境搭建，开发者需要安装 Node.js 和 npm 包管理器，这是开发 VS Code 插件的基础环境。随后，可以使用 VS Code 官方提供的 Yeoman 生成器来创建插件的骨架，快速生成基本的项目结构和文件。

2、编码阶段，开发者使用 TypeScript 或 JavaScript 编写插件的逻辑代码，并在 package.json 文件中定义插件的各项配置，如激活事件、贡献点（Contributes）和依赖项等。

3、调试和测试阶段，VS Code 提供了内置的插件调试工具，开发者可以在编写插件的同时进行测试和调试，确保插件的功能和性能符合预期。

4、打包和发布阶段，完成插件开发后，开发者可以使用 vsce 工具将插件打包为 vsix 文件，随后可以发布到 Visual Studio Code Marketplace 上，供其他用户下载和使用。

借助于 VS Code 的插件开发环境，本文将尝试开发出一款实用的共享终端插件。这款插件将允许用户邀请他人通过网络加入到自己的终端会话中，实现远程协作的功能。这将极大地促进开发团队之间的沟通和协作效率，为开发者提供一个强大的协作工具。

2.2.2 CoVsCode

CoVsCode 是一款基于 Visual Studio Code 的插件，它支持多个用户在 VS Code 中进行协作^[29]，并具有如下的优点：

1. 轻量级，快速响应。作为一款插件，它占用的系统资源较少，不会对用户的电脑造成较大负担。同时，它的响应速度极快，无论用户是在编写代码还是执行复杂的编程任务，CoVsCode 都能确保流畅的操作体验。这样，用户可以专注于工作，而无需担心插件的性能问题。
2. 拓展性强。它不仅能够与 Visual Studio Code 的其他插件顺利兼容，还可以无缝集成到用

户现有的工作流程中。这意味着用户可以根据自己的需求和喜好，自由选择并添加其他插件，打造出个性化的开发环境。

3. 兼容性好。基于 Visual Studio Code 平台，它可以在多个操作系统上运行，包括 Windows、MacOS 和 Linux。无论用户使用的是哪种操作系统，CoVsCode 都能提供一致的用户体验。这样，开发者可以在自己喜欢的平台上享受相同的开发体验，提高工作效率。
4. 易于开发，有成熟稳定的 API。它有着较为稳定的 API，使开发者可以在不改变原有代码的基础上进行深度定制和开发。这为开发者提供了更大的灵活性，使用户的项目能够更好地适应不断变化的需求。
5. 适用于多种编程语言。无论用户是在使用 Java、Python、C++ 还是 JavaScript，CoVsCode 都能提供强大的支持。这意味着用户可以在一个统一的环境中完成各种语言的编程工作，避免了频繁切换工具带来的不便。

2.2.3 GoLand

GoLand 是 JetBrains 公司开发的一款专为 Go 编程语言设计的集成开发环境（IDE）。GoLand 提供了全面的开发工具，包括代码高亮、智能代码补全、代码重构、调试器、版本控制集成等功能，这些功能能够极大地提高开发效率和代码质量。它支持常见的 Go 工具链，如 Go Modules、Dep、GolangCI-Lint 等，并且内置了对 Go 语法和标准库的深度支持。

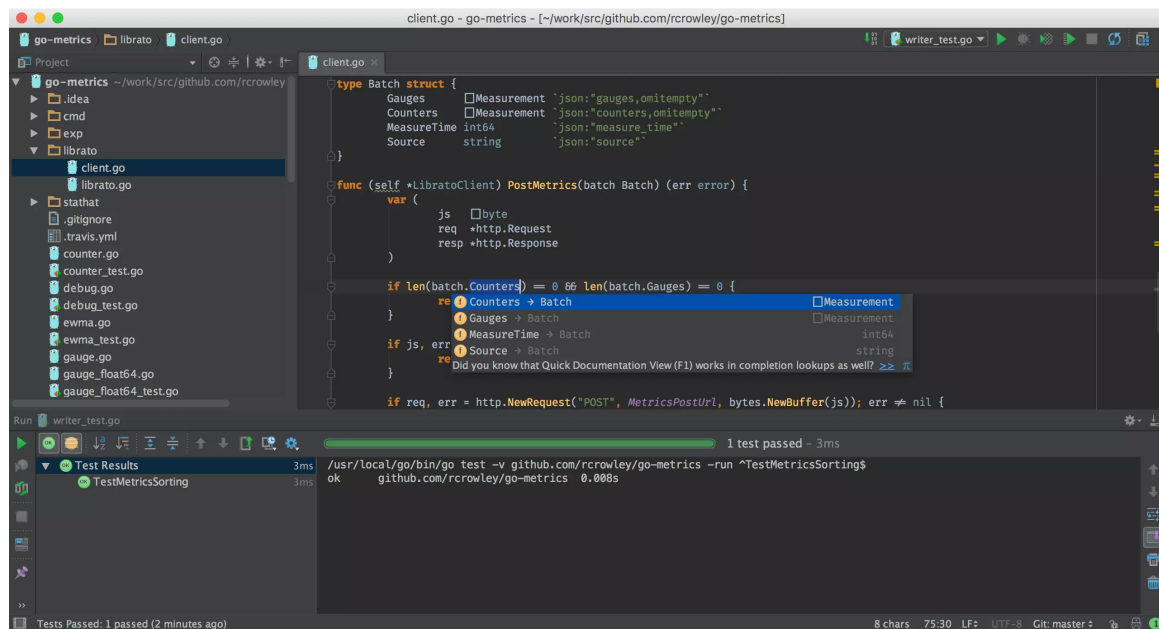


图 2.11 GoLand 使用示意图

GoLand 的用户界面简洁友好，设计上注重开发者体验。它还提供了一系列插件和扩展，允许用户根据自己的需求进行定制。此外，GoLand 还支持实时协作编程功能，使得多个开发者可以在同一个项目上共同工作，提高团队协作效率。GoLand 在设计上注重性能优化，能够处理大

型项目和复杂代码库，并保持良好的响应速度和稳定性，确保开发者能够高效地进行编码工作。此外，它还集成了代码质量管理工具，如静态代码分析工具（如 SonarQube）和代码检查工具（如 GolangCI-Lint），这些工具可以帮助开发者检测和修复代码中的潜在问题和缺陷，提高代码的可靠性和可维护性。本项目在涉及 Go 语言方面的开发使用该 IDE 为工具，其使用示意图如图 2.11 所示。

装
订
线

3 实时协同编程环境中的共享终端技术

3.1 现有共享终端方案的不足

目前，学术界已经有一些文章对共享终端的若干需求进行了研究与探索，例如 Tan 等人^[30]在论文中列出了共享终端的四个需求，包括的内容如表 3.1 所示。这些需求从适应性、保护机制、信息分享和权限管理等方面，全面探讨了共享终端的关键要素和实现路径，为相关研究和应用提供了重要参考。

表 3.1 Tan 等人调研得出的关于共享终端的四个主要需求

需求编号	需求描述	满足度	相关性
1	适应不同机器上的终端样式	62	0.59
2	用于在终端上访问本地文件的保护机制	73	0.77
3	分享更多信息而不仅仅是文本	80	0.88
4	用户自定义特定命令的权限	61	0.53

本文的目标是提出并设计一种新型的共享终端方案，旨在解决 Visual Studio Code Live Share 和 JetBrains Code With Me 等现有工具在共享终端功能上的局限。通过深入剖析这些协作工具的运作模式和权限设置，我们发现了两大问题，并针对性地提出了改进的方法。

首先，当前主流的协作工具，如 Visual Studio Code Live Share 和 JetBrains Code With Me，采用以主持人为核心的工作模式。这意味着会话高度依赖于宿主机的状态。一旦宿主机离线，整个会话将被迫终止，导致所有参与者被迫中断合作。此外，在这种模式下，仅主持人拥有创建新共享终端的权限，且这些终端完全基于主持人的电脑环境，其他参与者无法享有相应的创建和管理权限。

如图 3.1 所示，在 Code With Me 中，主持人可以选择发起终端的权限，如果选择 Private，那么创建的终端只有自己可以看到；如果选择 Shared，则系统会创建终端，权限则可以调整为完全只读或完全读写。

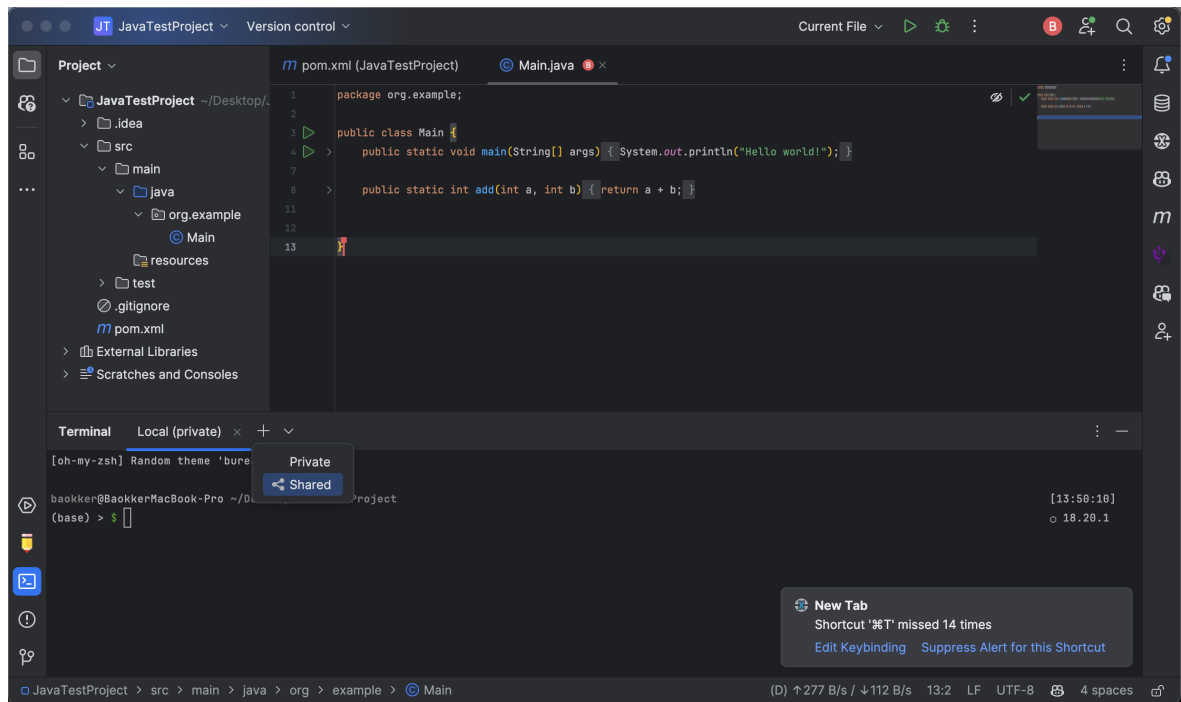


图 3.1 在 Code With Me 中，只有主持人可以发起共享终端

然而，如图 3.2 所示，参与者在创建终端时，只能选择本地的 `bash` 终端，并没有权限创建共享终端。当协作中出现需要使用参与者的终端的情况时，便会影响到团队的效率。

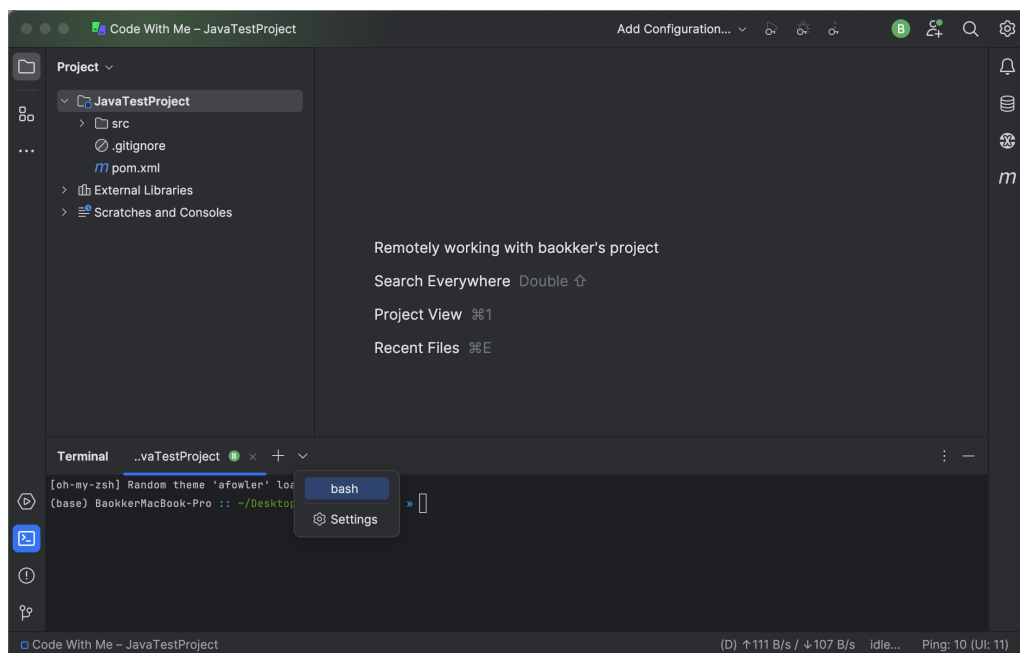


图 3.2 在 Code With Me 中，参与者无法创建共享终端

其次，现有共享终端的权限设置存在明显缺陷。只读权限虽适用于教学场景，但在需要深度协作的环境中，其限制性过强，难以满足合作需求。而完全读写权限虽提供了更大的灵活性，但也可能带来安全风险。参与者可能误删关键文件，访问或修改敏感数据，甚至可能面临恶意攻击的风险。以 Code With Me 为例，它在协作时在共享终端方面仅支持完全只读（View only）和完全读写（Full access），这意味着权限的分配可能不够灵活。

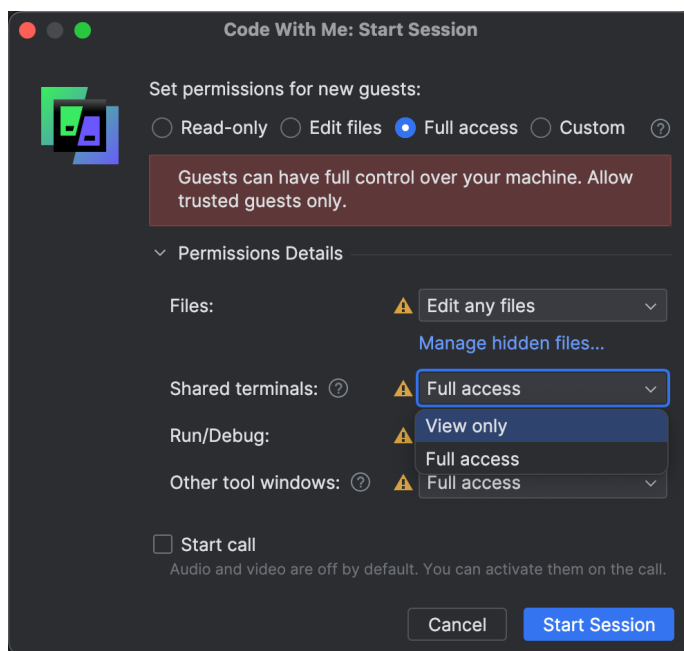


图 3.3 JetBrains Code With Me 的权限设置

例如在图 3.3 中，当发起者通过 Code With Me 创建 Session 时，在 Shared terminals 选项中，只有 View only（完全只读）和 Full access（完全读写）两个选项。当若干位程序员通过共享终端协作时，如果采取完全只读的共享终端策略，此时只有宿主机可以操作，其他程序员无法输入任何字符，这将影响该程序员的协作效率；而当采取完全读写的共享终端策略时，另一名程序员的权限则过大，所有参与者都拥有相同的权限，这不仅增加了误操作的风险，也给系统安全带来了隐患。例如，一名用户可能因为不熟悉环境而意外删除重要文件，或者在无意间修改了关键配置，从而导致系统崩溃。此外，恶意用户可能利用这一权限对系统进行攻击，获取敏感信息，甚至破坏系统的正常运行。如图 3.4 所示，该程序员甚至可以直接访问系统根目录，这无疑对宿主机的安全性造成了威胁。

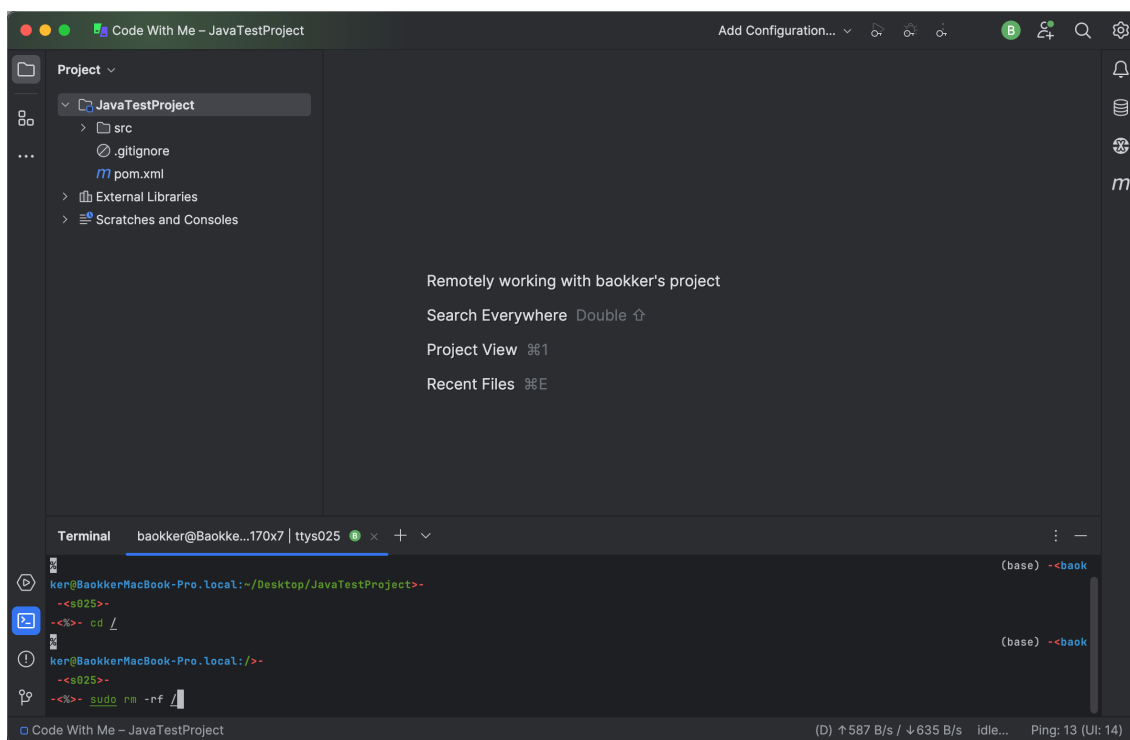


图 3.4 在完全读写策略下，参与者的操作可能对宿主机造成威胁

尽管开源社区中存在一些如 Upterm 等优秀的软件，但它们仍存在一些不足。例如，这些软件在支持共享终端的同时，可能无法很好地支持代码编辑，从而影响了协作效率。Upterm 等共享终端软件在设计上主要侧重于提供一个可以远程访问的终端环境，这对需要实时监控系统状态或进行简单指令输入的场景非常有用。然而，在复杂的开发环境中，代码编辑是一个核心需求，许多开发者需要在协作时进行代码编写、调试和实时修改。而 Upterm 在这一点上显得力不从心，其终端共享功能虽然强大，但对代码编辑的支持有限，缺乏像现代代码编辑器（如 VS Code 或 Sublime Text）所具备的智能提示、语法高亮和代码格式化等功能。这使得开发者在进行深度协作时需要频繁切换工具，降低了工作效率。

为了解决上述问题，我们提出了一种改进的共享终端插件方案。该方案旨在构建一个更加灵活、无主持人依赖的共享终端系统。即使在主持人离线的情況下，其他参与者也能继续维持会话的进行。在权限管理方面，我们计划引入更加精细和安全的权限设置，以满足不同场景下的协作需求，同时确保数据的安全性。

在技术实现上，我们将利用 VS Code 的插件开发环境，结合现代网络编程和安全技术，打造一款稳定、高效的共享终端解决方案。这款插件将支持跨平台运行，提供丰富的自定义选项，旨在为用户带来卓越的协作体验，同时显著提升远程团队的协作效率。

3.2 无约束的多层次权限安全共享终端方案（FlexSecure）

为了应对当前共享终端软件中的挑战，笔者经过深入分析和研究，提出了一种新型的无约束

的多层次权限安全共享终端。考虑到该方案兼顾了共享终端的灵活性和安全性，因此将其命名为 FlexSecure。

首先，FlexSecure 方案针对会话高度依赖宿主机状态的问题，采取了一种更加平等的协作方式。在这一新模式下，不再仅由会议主持人掌控会话的启动权，而是赋予每个参与者启动新共享终端会话的权限。这些共享终端可以在参与者的个人计算环境中灵活运行，而不再受限于主持人的电脑环境。这样一来，每位参与者都能够在自己熟悉且优化的环境中发挥作用，从而显著提高整个团队的协作效率和灵活性。

其次，为了增强共享终端软件与实时协同编程工具之间的协作，该方案将共享终端与实时协同编程紧密结合。这项功能允许参与者在编写代码的同时，在命令行界面中进行无缝协作，实现编程和指令输入等操作的并行处理，无需频繁切换应用界面。这一改进不仅促进了团队成员之间的沟通，还显著提升了协作效率和编程体验。

最后，FlexSecure 方案针对共享终端软件可能存在的安全风险，实施了一系列安全升级措施。除了现有的完全只读和完全读写权限模式外，该方案引入了更精细的权限控制功能。这一新功能依据预设规则，智能地排除特定指令的执行权限，对存在风险的指令进行警告或禁止，从而在不干扰正常协作的前提下，有效保护发起终端者的系统安全和个人隐私。通过这些安全增强措施，用户将能够获得一个更加安全可靠的共享终端环境。

3.3 FlexSecure 方案流程

3.3.1 更加灵活的共享终端

在改进方案中，实时协作编程将在分布式的协作工具中进行，所有参与者都可以按图 3.5 的流程进行操作。首先，程序员在其计算机上启动共享终端，生成一个 SSH 访问链接，该链接会传输到 CoVsCode 的服务器（Server），随后广播给其他成员，其他成员接收到链接后可以通过 CoAction 进行 SSH 访问，从而实现灵活高效的协作。这一流程的改进之处在于，所有程序员都可以启动共享终端，从而大大提升了协作的灵活性。

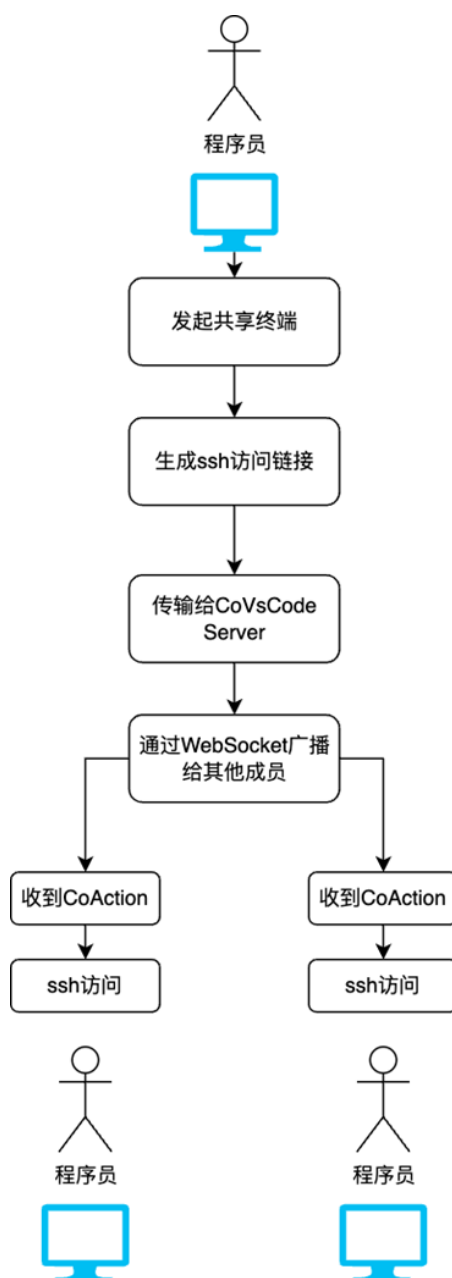


图 3.5 FlexSecure 共享终端流程图

3.3.2 更加安全的协作方式

在 FlexSecure 方案中，笔者设计了两种适用于实际情况的模式：只读模式（Read Only）和安全模式（Read Only）。在只读模式下，参与者无法输入终端，只能观看；在安全模式下，参与者

的部分存在风险性的命令会被限制，其流程图如图 3.6 所示。FlexSecure 权限控制的流程如下：首先，发起者被允许执行操作；参与者则需通过一系列检查。参与者进入流程后，首先检查是否为只读模式。如果是，则不允许输入；如果不是，则继续检查是否为危险指令（如 `rm` 删除命令会将发起共享终端者的计算机上的文件删除，很可能会威胁到计算机的安全，只能由发起者自己来执行，不应当由其他人来执行）。如果是危险指令，禁止执行并弹出通知；如果不是，再检查是否为风险指令（如 `mv` 移动命令，在多数情况下并不会影响到主机的安全，所以没有必要拦截；但考虑到潜在的风险性，仍然应当有必要让发起者及时地获得通知）。如果是风险指令，允许执行并弹出通知；否则，允许执行。这一流程确保了系统操作的安全性，防止了危险或不必要的指令执行。

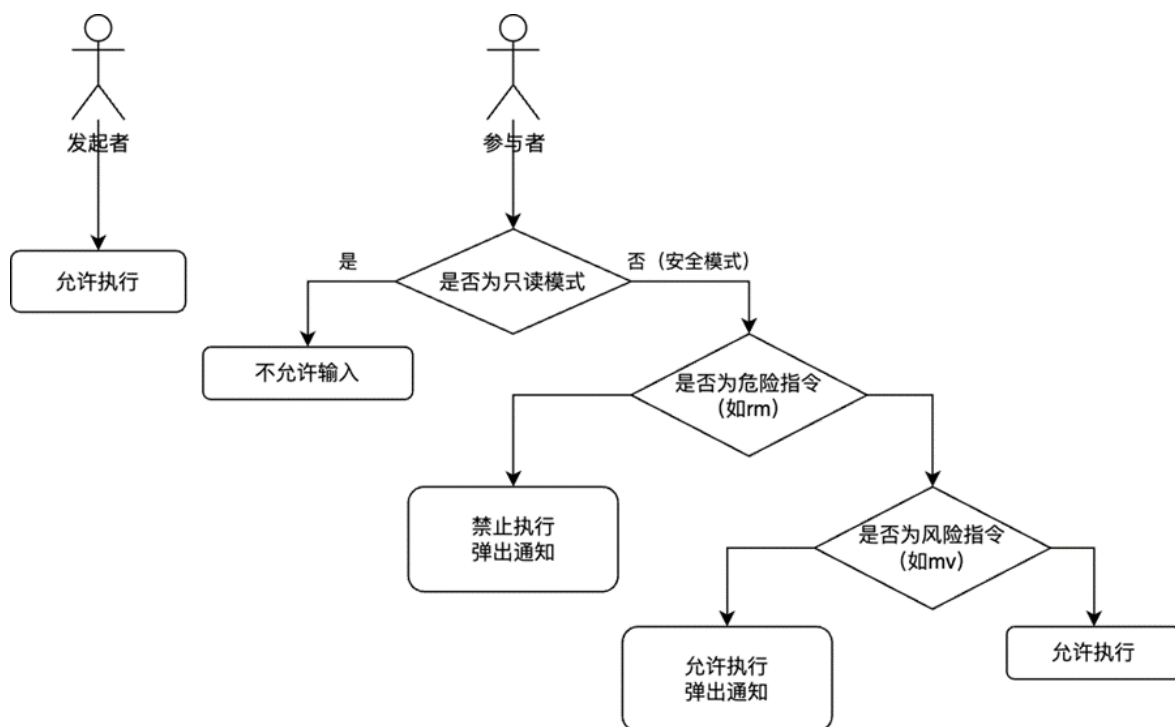


图 3.6 FlexSecure 权限控制的流程图

4 系统设计与实现

4.1 需求分析

4.1.1 功能性需求

考虑到本项目的软件需求，我们对功能需求进行了全面分析和规划，以确保系统能够满足程序员之间高效、安全的协作编程需求。以下是具体的功能需求：

1. 实时协作编程与共享终端相结合

为了满足实时协作编程的需求，系统必须支持程序员在代码层面进行实时同步和编辑。这意味着，团队成员可以在同一个代码文件中同时进行修改和查看，所有更改将被即时反映。同时，结合共享终端的功能，参与者可以在终端界面上进行实时共享和交互，提高协作效率。这种结合将使得团队成员无论身处何地，都能实时参与到编程工作中，减少沟通成本和等待时间，确保项目进度的顺利推进。

2. 更加平等的共享终端协作方式。

除了作为主持人的程序员拥有发起实时协作编程的权限外，其他参与者也应当具备同样的权限。这意味着任何团队成员都可以根据自己的需要创建共享终端，并邀请其他成员加入协作。这种平等的协作方式有助于激发团队成员的积极性和创造力，促进更加高效的团队合作。同时，这种权限分配能够减少主持人的负担，确保团队每个成员都能在自己的最佳状态下进行工作，提高整体协作效率。

3. 更精细的权限控制

为了确保系统的安全性和隐私性，我们需要实现更精细的权限控制功能。除了常见的只读和读写权限外，系统还需要提供更加灵活的权限管理方法。通过适当限制参与者的权限，我们可以确保一些涉及安全和隐私的操作只能由发起共享终端者本人执行。同时，这种权限控制方法不会对正常的协作产生负面影响，保证了团队成员之间的顺畅沟通和协作。

需求的用例图如图 4.1 所示。

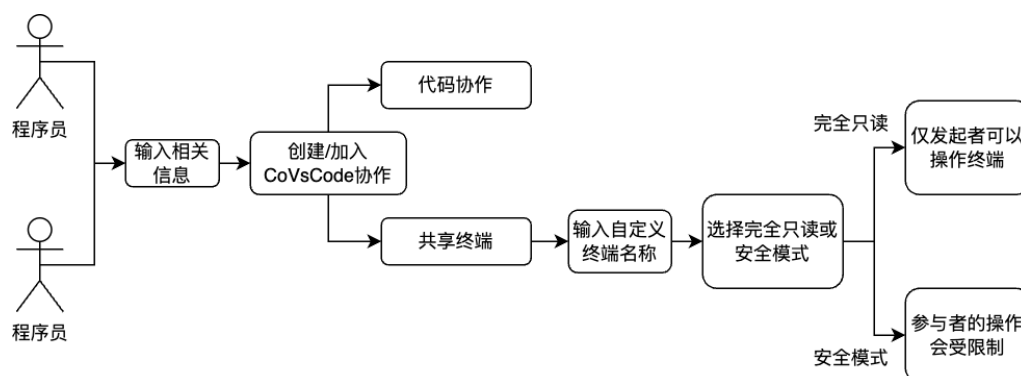


图 4.1 基于 CoVsCode 的共享终端流程示意图

4.1.2 非功能性需求

1. 可用性

系统需要设计直观易用的用户界面，使用户在初次使用时就能迅速掌握基本操作，降低学习成本。同时，提供详细的错误提示和友好的帮助信息，使用户在遇到问题时能够快速找到解决方案，提升用户体验。

2. 性能要求

为了保证系统在不同网络环境下的稳定表现，需要定期进行性能测试，并根据测试结果进行优化。此外，优化数据传输机制，减少网络延迟和数据丢包，确保用户之间的协作能够流畅无阻。

3. 兼容性

共享终端插件需要确保与 VS Code 的各个版本保持兼容，避免因版本升级导致插件失效。同时，插件需要支持多种编程语言和终端环境，如 `bash`、`zsh` 等，以满足不同用户的工作需求。确保插件在各种操作系统和硬件平台上都能稳定运行，也是兼容性的重要方面。

4. 安全性

系统需要引入会话超时机制，确保非活动用户在设定时间后自动断开会话，防止未授权访问。此外，在软件开发生命周期的每个阶段都应考虑安全因素，遵循安全编码标准，以减少潜在的安全风险。

5. 可扩展性

为了满足不断增长的用户需求和并发会话数，系统需要具备良好的可扩展性。通过优化系统架构和资源配置，确保在用户量增加时仍能保持稳定的性能表现。

4.2 关键设计与实现

4.2.1 在 CoVsCode 中实现共享终端

为了不再局限于主持人，实现在 CoVsCode 中进行无约束的终端共享，我们结合了开源项目 Upterm 以达到该功能。CoVsCode 使用 WebSocket 进行信息交流，具有迅速、低功耗等特点。WebSocket 是一种全双工通信协议，它允许客户端和服务端之间的实时数据交换，极大地减少了通信延迟，提高了用户体验。而 Upterm 通过 SSH 进行连接，具有安全性和加密性的特点。SSH（Secure Shell）是一种加密网络协议，用于在不安全的网络上安全地操作网络服务，确保数据传输的安全性和保密性。

在用户输入相关信息后，系统将创建一个新的终端，并通过 Upterm 发起协作。Upterm 将会生成一条可用于访问的 SSH 链接，该链接通过 CoVsCode 的 WebSocket 通信，封装为 CoAction 发送给除发起者外的所有人。这些用户收到 CoAction 后也将建立新的终端，并通过 SSH 加入到共享终端中进行协作。整个过程通过高效的 WebSocket 通信和安全的 SSH 连接，确保了协作的及时性和安全性。

CoAction 的类结构如表 4.1 所示，wsMsgDataType 表示信息的数据类型，以便服务端收到后根据不同类型分别做出处理；_user 则用来标示该行为的发起者。

表 4.1 CoAction 的类结构

属性	类型	描述
wsMsgDataType	WsMsgDataType	信息的数据类型
_user	CoUser	发送的用户

而 TerminalBeginAction 则是 CoAction 的继承类，它在原有的基础上，添加了三个新的属性，其中 repoId 表示当前协作项目的唯一 ID，terminalName 表示发起者定义的终端名称，sshPath 表示访问的路径。TerminalJoinAction 的类结构与 TerminalBeginAction 类似，主要用于其他参与者收到信息后进行处理。两个类的类结构如表 4.2 所示。

表 4.2 TerminalBeginAction 与 TerminalJoinAction 的类结构

属性	类型	描述
repoId	string	当前所处项目的 ID
terminalName	string	自定义终端的名称
sshPath	string	SSH 访问路径

服务端在收到 TerminalBeginAction 后，将处理该行为，并发送给除发起人之外的其他所有项目协作者。具体流程为：服务端从 action 中提取 repoId 和 user 信息，在获取所有用户的键后，移除发送该行为的用户，并创建新的 TerminalJoinAction。最后，服务端向所有其他用户发送这个 TerminalJoinAction。处理算法如下所示。

Algorithm 1 Start Sharing Terminal (Server Side)

Input: action containing terminal sharing details

```

let repositoryId = action.user.repoId
let repository = this.repoManager.getRepoById(repositoryId)
let user = action.user

// Initialize an empty list to store keys
let keys = []

for (each entry in repository.userMap)
    keys.add(entry.key)

// Remove the user who sent the action from the keys list
keys = keys.filter(eachSid => eachSid != user.siteId)
    
```

```
// Create a new TerminalJoinAction object with the specified properties
let terminalJoinAction = new TerminalJoinAction({
  sshPath: action.sshPath,
  terminalName: action.terminalName,
  type: 'terminalJoinAction'
})

for each key in keys
  this.sessionManager.sendActionById(key, terminalJoinAction)
```

其他客户端在收到服务端发送的 `TerminalJoinAction` 后，将显示提示信息，创建并显示终端，并发送 SSH 路径到终端，从而实现共享终端协作。处理算法如下所示。

Algorithm 2 Start Sharing Terminal (Client Side)

Input: action containing received terminal sharing details

```
displayMessage('Join to share terminal');
let terminal = createTerminal(action.terminalName);
terminal.show();

if (action.sshPath is undefined)
  terminal.sendText('echo "there is something wrong..."');
else
  terminal.sendText(action.sshPath);
```

由于 CoVsCode 采用平等而非主从的协作模式，所有用户都有权发起共享终端，而在 Live Share、Code With Me 等软件中，只有主持人有权发起共享终端。这种设计提升了协作的灵活性和公平性，使团队成员之间的协作更加高效。

4.2.2 共享终端中的细化权限控制

在 FlexSecure 共享终端中，我们设置了两权限控制模式：完全只读（Read Only）和安全模式（Safe Mode）。

完全只读模式与主流共享终端的模式类似，只有发起者可以操控自己的终端，其他人可以一起观看但无权操作。这种模式适用于展示和教学场景，确保演示者的控制权。

安全模式改进了完全读写模式，在该模式中，系统将读取每一个客户端的输入流，并设置中间处理逻辑，不断获取用户输入。当收到表示命令结束的字符（如 `\r`、`\n`）时，判断输入指令并设

计算法进行检验。如果指令中包含一些可能存在安全隐患的指令（例如，mv 移动文件，cp 复制文件），则系统不会停止该命令的执行，但是会在命令和系统通知中给予提示；如果指令中包含危险命令（例如，rm 删除文件，cd /移动到系统根目录等），则停止该命令的运行，同时发起通知提示。这样既保护了系统的安全性，也避免了可能的隐私侵害。此模式允许用户在协作中进行操作，但同时确保了安全性，适用于需要多方共同操作但又需严格控制命令执行权限的场景。

为了实现该功能，我们将处理终端的输入流，以便对输入的命令进行判断。具体的代码逻辑如下：首先，我们创建一个新的上下文读取器，用于读取用户会话的输入。接下来，我们进入一个无限循环，以字节为单位读取输入流。每次读取时，我们将输入转换为字符串，并根据内容进行处理。具体排除算法如下：

Algorithm 3 Exclude Risky Commands

Input: Input stream `reader` from the terminal

```
let currentCommand = "";

while (true) {
  let input = getInput();
  if (input.includes('backspace') && currentCommand) {
    currentCommand = removeLastCharacter(currentCommand);
  } else if (input.includes('ctrlC')) {
    currentCommand = "";
  } else if (input.includes('enter') || input.includes('newline')) {
    if (isDangerousCommand(currentCommand)) {
      sendCtrlC();
      notifyClientDanger();
      showDangerNotification();
      currentCommand = "";
      continue;
    } else if (isWarningCommand(currentCommand)) {
      writeToTerminal(buffer.subarray(0, bytesRead));
      notifyClientWarning();
      showWarningNotification();
      currentCommand = "";
      continue;
    }
    currentCommand = "";
  } else {
    currentCommand = currentCommand + input;
  }
}
```

```
currentCommand = appendInput(currentCommand, input);  
}  
  
writeToTerminal(buffer.subarray(0, bytesRead));  
}
```

装
订
线

5 系统运行与测试

5.1 插件的安装与运行

本节将详细描述插件的安装过程以及插件的运行方法。

5.1.1 插件安装

首先，安装该共享终端插件前，需要用户已安装最新版本的 Visual Studio Code 编辑器。

可以前往 <https://code.visualstudio.com/download> 官方网站页面进行下载和安装（如图 5.1 所示）。

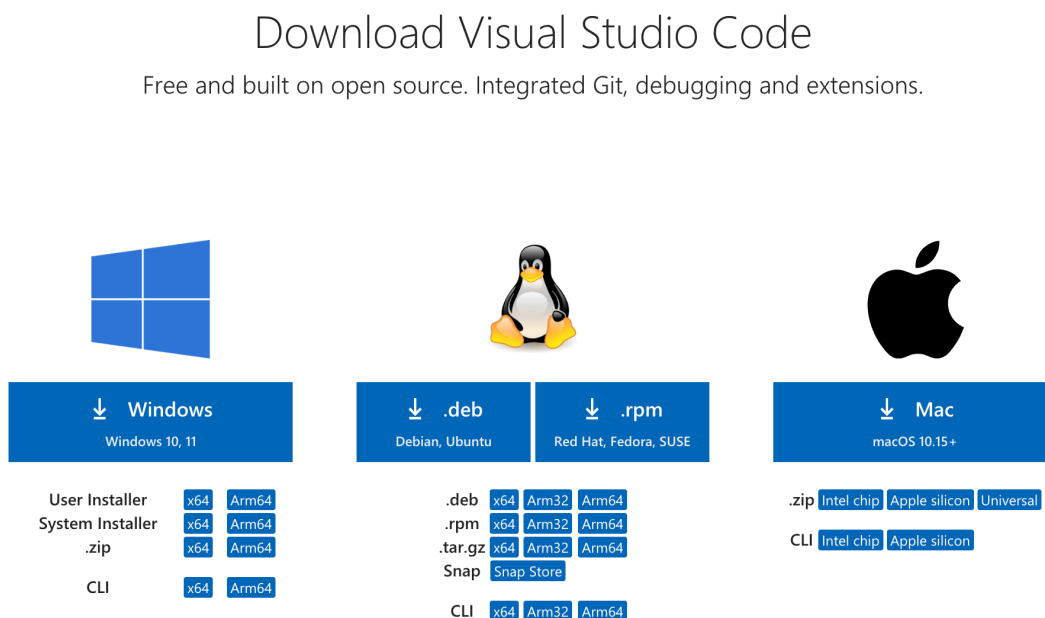


图 5.1 Visual Studio Code 官方下载页面示意图

在 VS Code 的侧边栏中点击扩展图标（或使用快捷键 **Ctrl+Shift+X**）。在扩展管理器中点击"更多"菜单，选择"从 VSIX 安装"（如图 5.2 所示）。

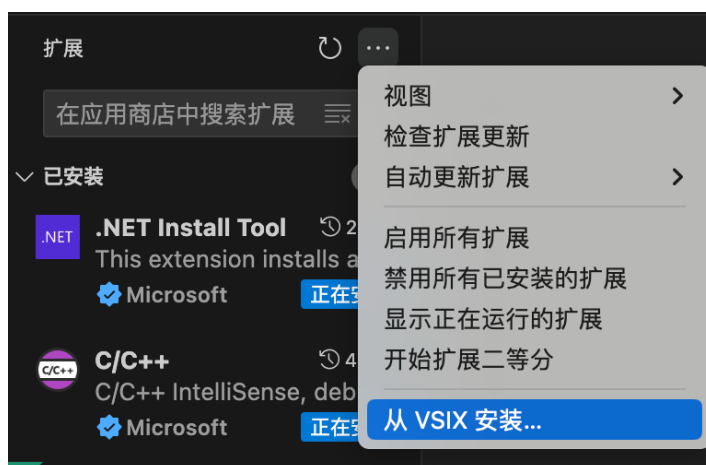


图 5.2 通过 VSIX 安装 CoVSCode 示意图

选择已导出的 CoVsCode 的 VSIX 文件，进行安装。安装完成后可能要求重启 VS Code。如果可以在插件列表中找到 CoVsCode（如图 5.3 所示），则说明已经安装完成。

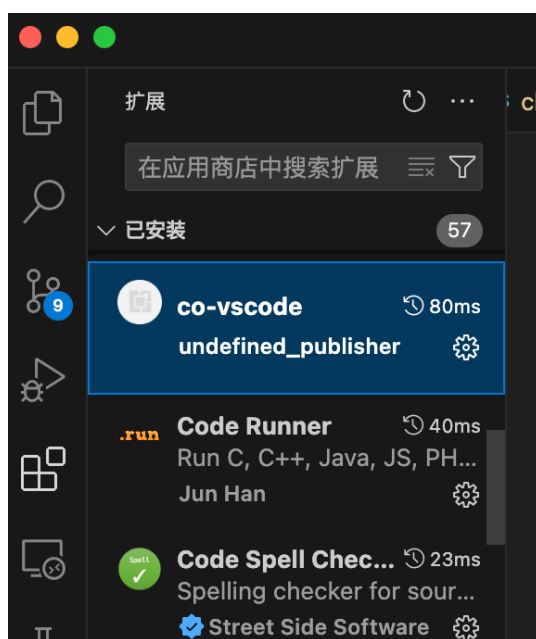


图 5.3 CoVsCode 插件安装完成示意图

此外，系统如果需要通过 FlexSecure 共享终端功能，需要下载并安装 Go，以确保共享终端的功能可以顺利执行。可以前往 <https://go.dev/doc/install> 官方网站下载并安装 Go（如图 5.4），设置 GOPATH 等环境变量，并通过以下命令安装：

```
git clone https://github.com/Baokker/upterm.git
```

```
cd upterm
```

```
go install ./cmd/upterm/...
```

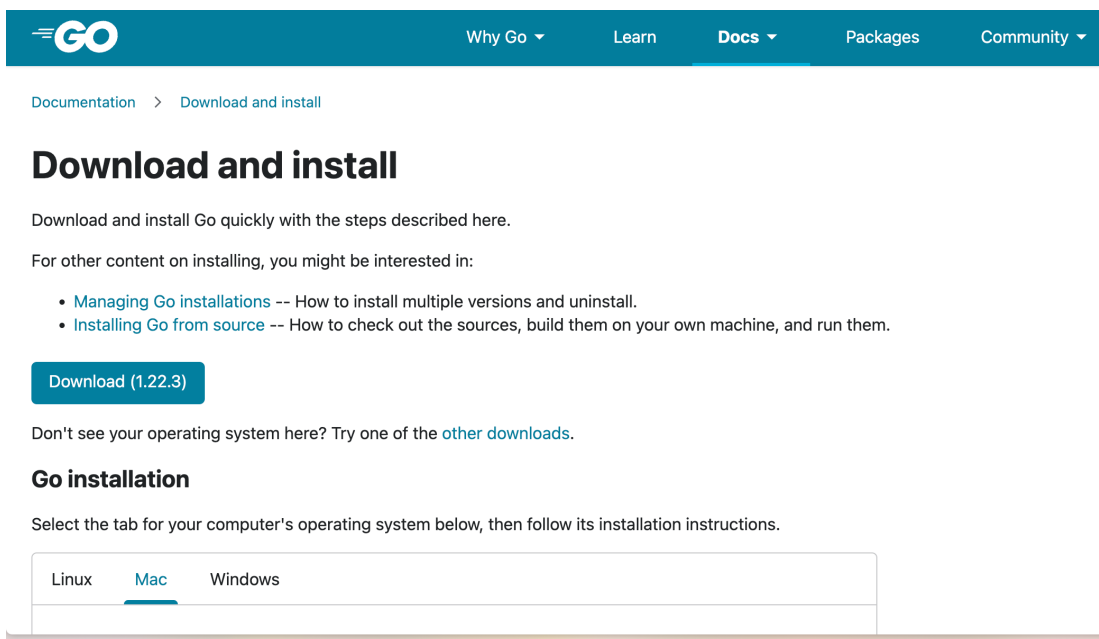


图 5.4 Go 官方下载页面示意图

5.1.2 插件运行

插件的总体运行图如图 5.5 所示：

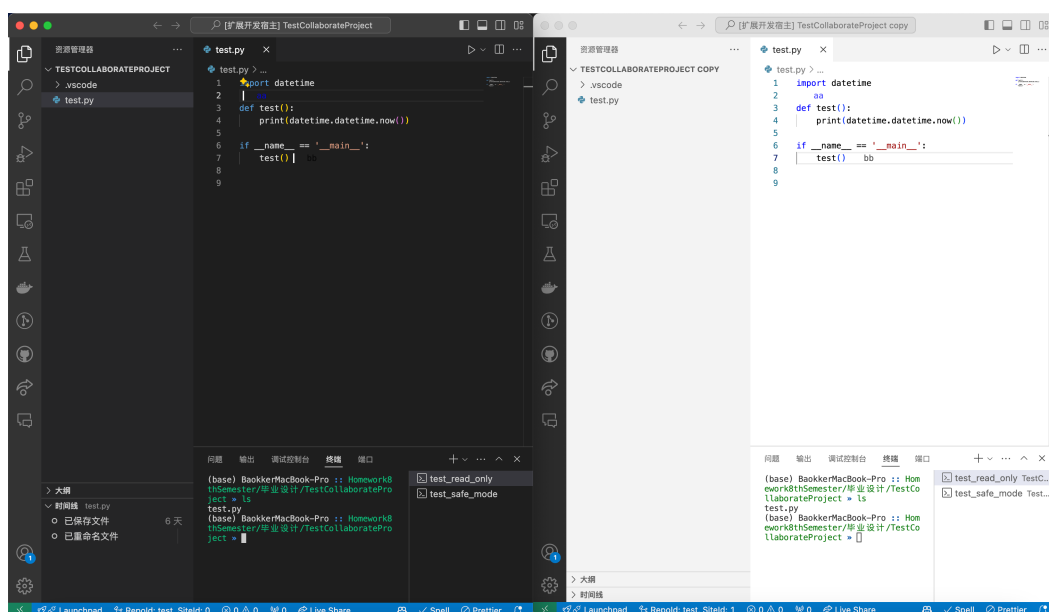


图 5.5 插件总体运行图

安装插件之后，如正常激活，左下角会出现 CoVsCode 字样（如图 5.6），用户可以点击左下角的该文字启动插件，此时会初始化一个新的共享终端会话。

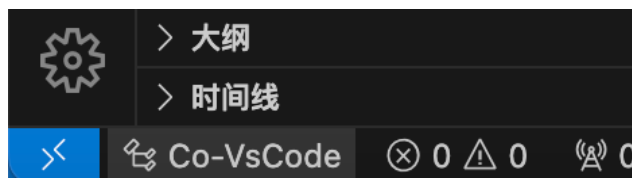


图 5.6 左下角的 CoVsCode 图标

点击以后，会出现“新建仓库”和“加入仓库”两个选项。当第一次创建仓库时，用户可以点击“新建仓库”选项，如图 5.7 所示。

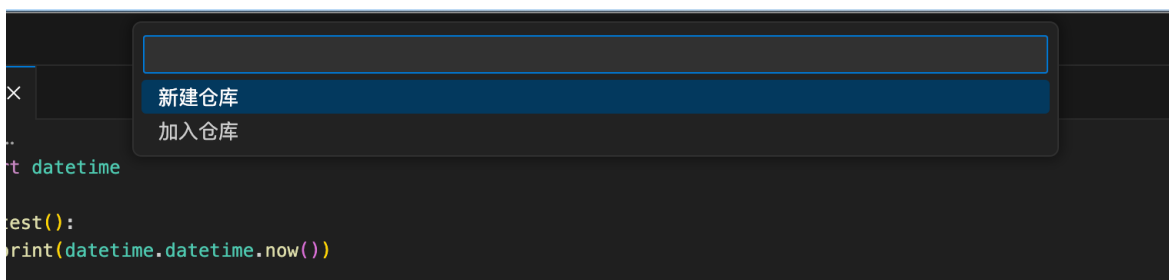


图 5.7 用户选择新建仓库

在接下来的弹窗中，用户输入仓库的 Websocket 地址，默认为本地 localhost，端口为 3000，如图 5.8 所示。

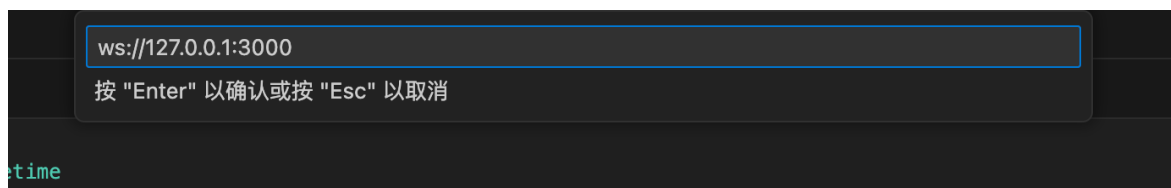


图 5.8 用户输入仓库地址

在接下来的弹窗中，用户输入自己在该仓库中的用户名（如图 5.9 所示）。

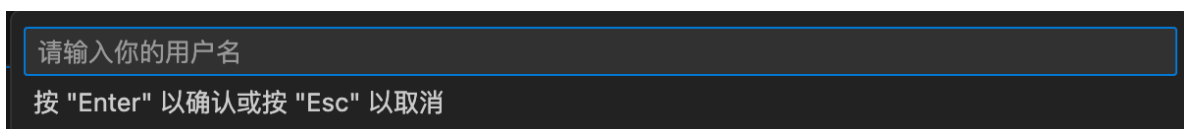


图 5.9 用户输入自己在该仓库中的自定义用户名

在接下来的弹窗中，用户输入自定义的仓库 ID（如图 5.10 所示）。

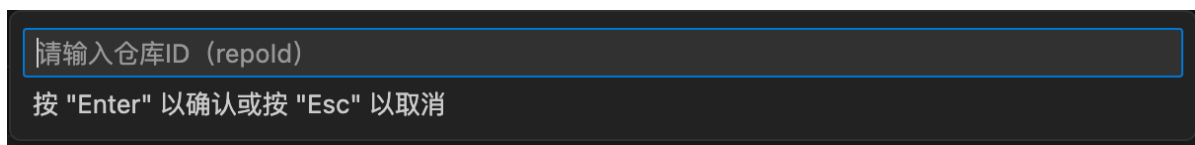


图 5.10 用户输入自定义仓库 ID

成功连接后，鼠标悬浮在左下方，可以看到如图 5.11 所示的 RepoId, SiteId 和 Current Users（当前用户），这表明已经该用户已经成功创建 Repo。

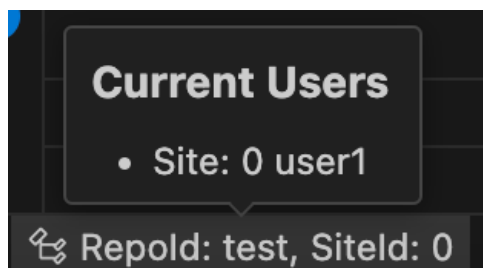


图 5.11 成功建立 Repo 示意图

其他用户同样可以点开左下角的 CoVsCode，选择加入仓库以和他人协作（如图 5.12）。

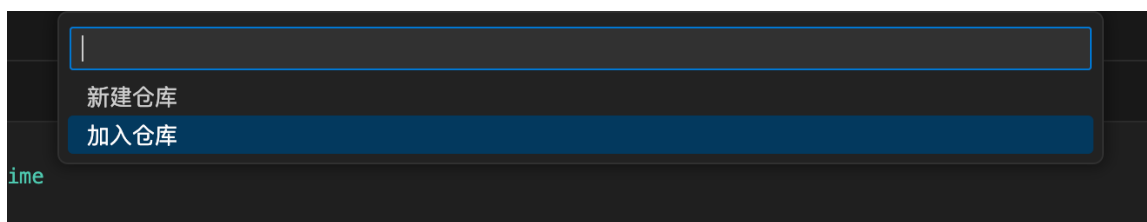


图 5.12 参与者选择加入仓库

选择以后，如果当前工作区非空，系统会给出提示，询问用户是否需要清空该工作区以避免文件冲突（如图 5.13）。

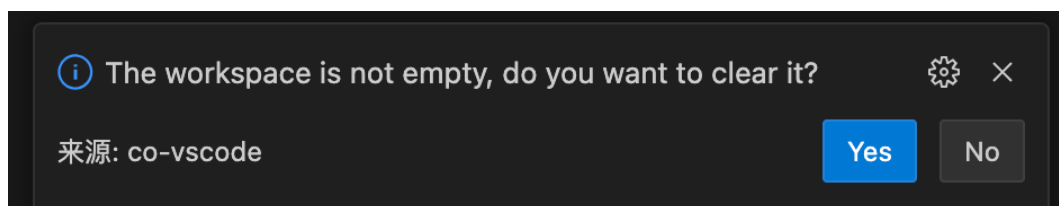


图 5.13 系统给出是否清空当前作业区的提示

与上文创建 Repo 的过程类似，加入者在输入仓库地址，用户名和仓库 ID 后，可以成功加入该作业区，文件可以自动同步（如图 5.14）。

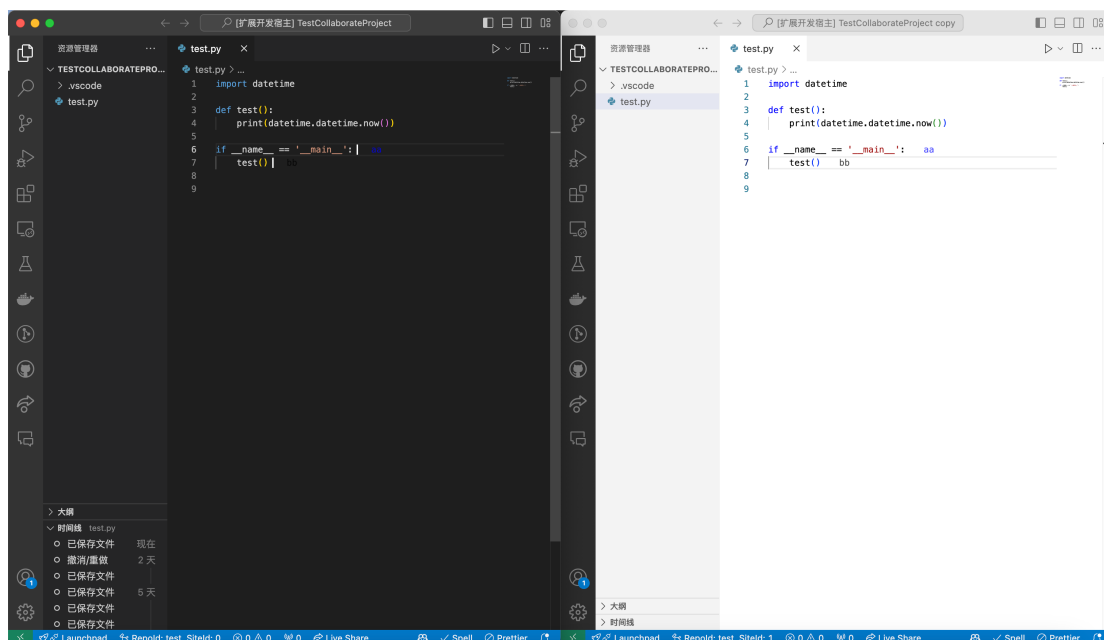


图 5.14 参与者加入后，文件将自动同步

任意一名用户都可以点击左下角的位置，选择“开始共享终端”来进行共享（如图 5.15）。

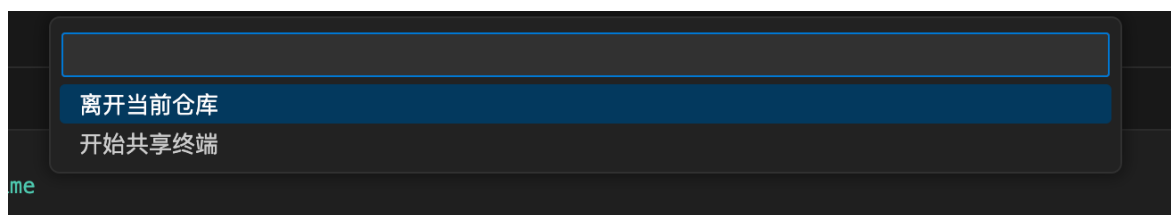


图 5.15 用户选择开始共享终端

输入自定义的终端名称（如图 5.16）后，用户还需要选择终端的共享形式（如图 5.17），其中 read-only 表示完全只读，safe mode 表示安全模式。

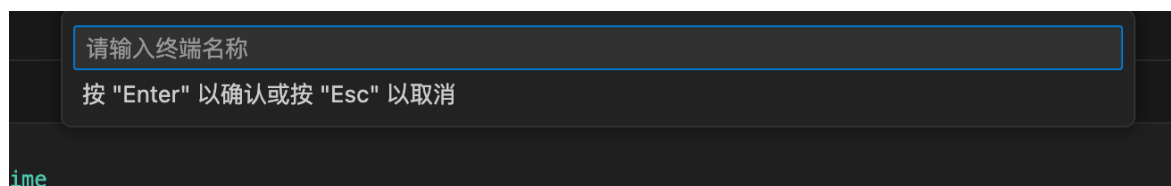


图 5.16 系统提示用户输入终端名称

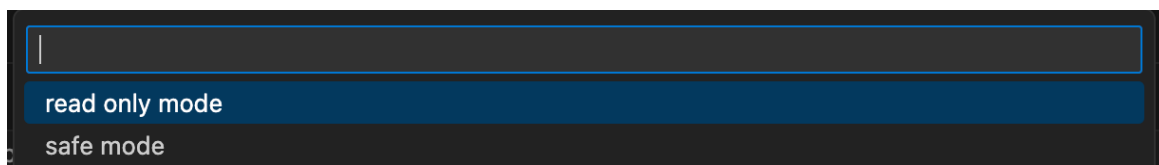


图 5.17 系统提示用户选择完全只读或安全模式

当其他用户成功加入该共享终端后，系统会给出通知，如图 5.18 所示。

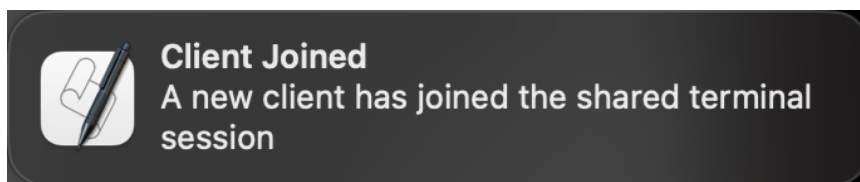


图 5.18 系统提示有用户加入

在完全只读模式中，参与者只能观看，不能输入指令，如图 5.19 所示。

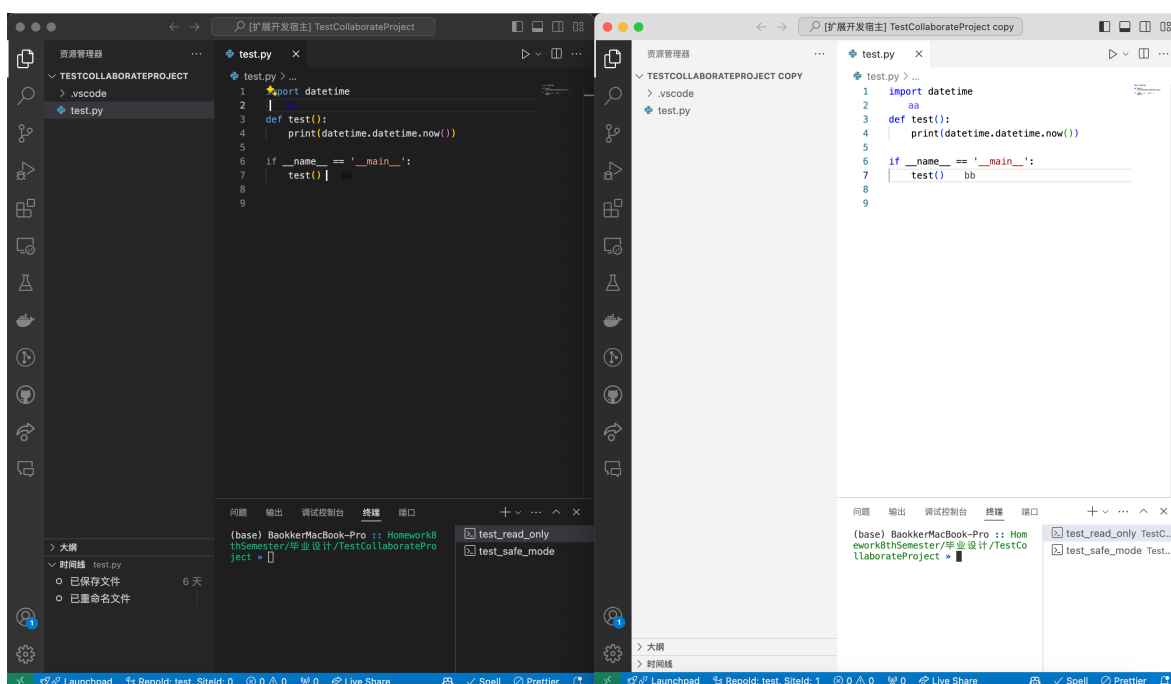


图 5.19 完全只读模式示意图

在安全模式中，一些可能会破坏宿主机的危险指令将会被警告或禁止执行。

如图 5.20 所示，当其他用户试图执行 `mv test1.py test.py` 指令重命名文件时，系统会给予通知提示，并弹出通知框提示用户，但并不会阻止该操作的进行。通过这样的方式，系统可以使协

作者感知到一些命令的潜在风险，提高安全性。

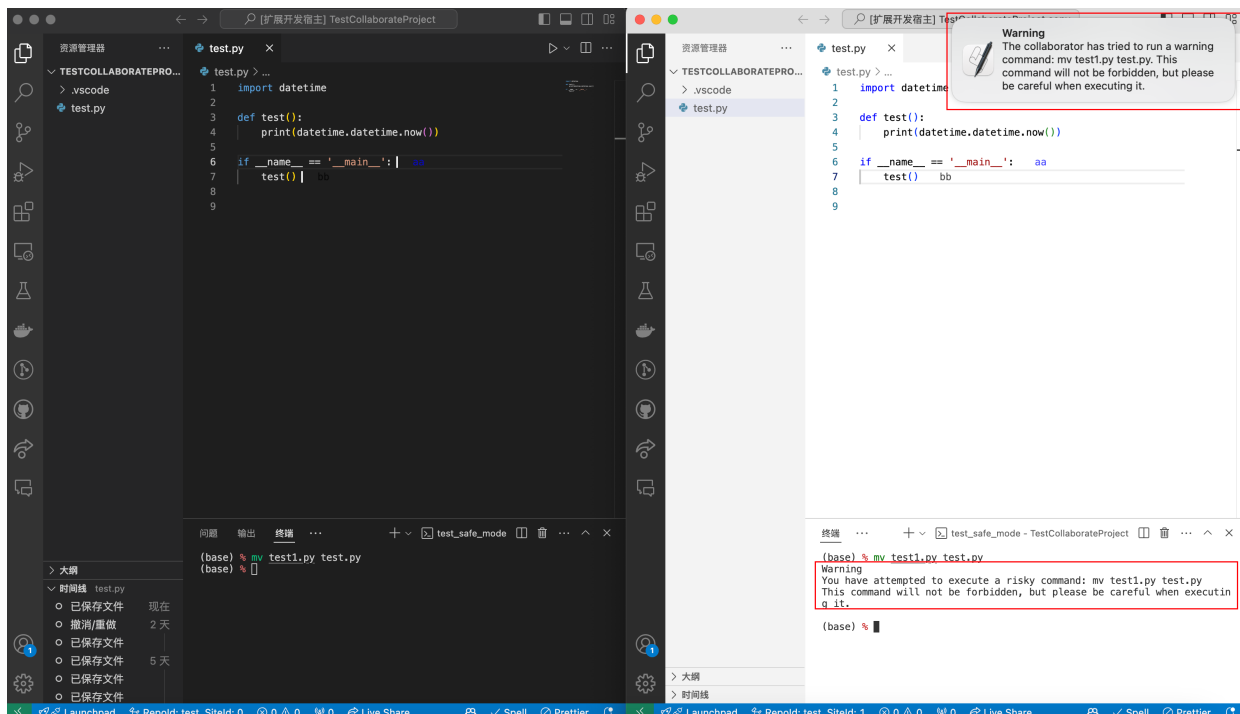


图 5.20 安全模式下，系统对于风险指令提出警告，并弹出提示

如图 5.21 所示，当其他用户试图执行 `rm test.py` 指令删除文件的操作时，系统会自动阻止该指令的执行，并弹出通知框提醒用户该操作可能存在危险。通过这种方式，系统有效地防止了因误操作或恶意指令带来的潜在风险，确保了协作环境的安全和稳定。同时，提醒机制也有助于用户了解哪些操作是被限制的，从而培养更安全的操作习惯。

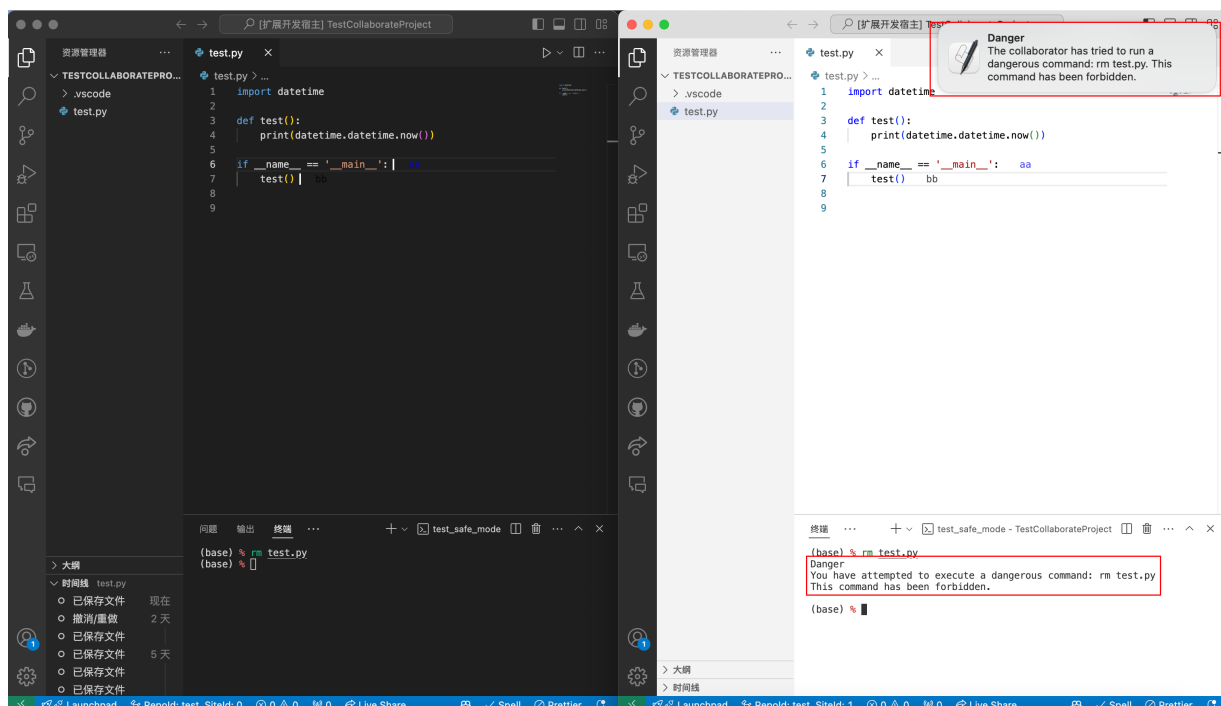


图 5.21 安全模式下，系统阻止危险指令，并弹出提示

参与者可以通过点击右下角的 CoVsCode，选择离开当前仓库来结束合作（如图 5.22）。

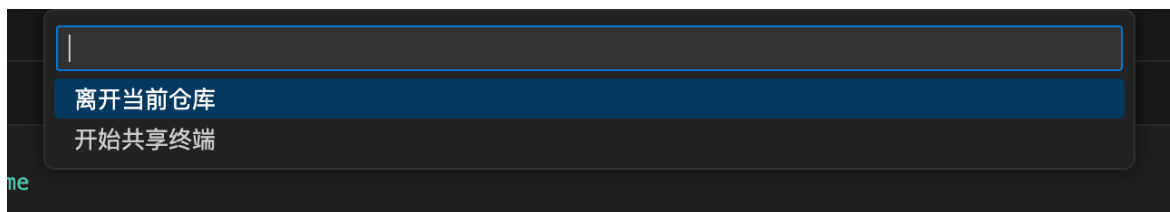


图 5.22 用户选择离开当前仓库

当参与者离开该共享终端后，其他人会收到通知，如图 5.23 所示。

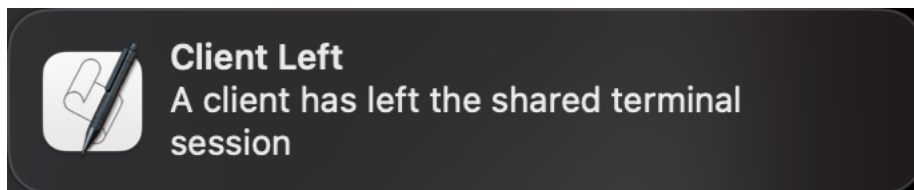


图 5.23 其他用户收到离开共享终端的通知

5.2 综合测试

为了全面测试 FlexSecure 终端，我们邀请了 10 名程序员使用该原型系统，并对他们进行了详尽的用户调研。在调研过程中，10 位程序员被分成不同的协同小组，分别在不同的协同工作区中展开工作。这些小组的工作结束后，我们使用 SUS（System Usability Scale）问卷对他们进行了调查。SUS 是一种广泛应用的用户体验评估工具，主要用于衡量和评估软件、网站及应用程序等系统的可用性，其标准问卷如表 5.1 所示^[31]。

表 5.1 参与调研的程序员的合作情况

人数	组数	平均协同编程时长/分钟
2	3	15
3	2	15
5	1	20

调研的具体实施过程为：每位程序员在参与分组并完成指定的编程任务后，需填写设计好的 SUS 问卷（如图 5.25 所示）。所有问卷完成后，我们对问卷结果进行细致的整理和分析。

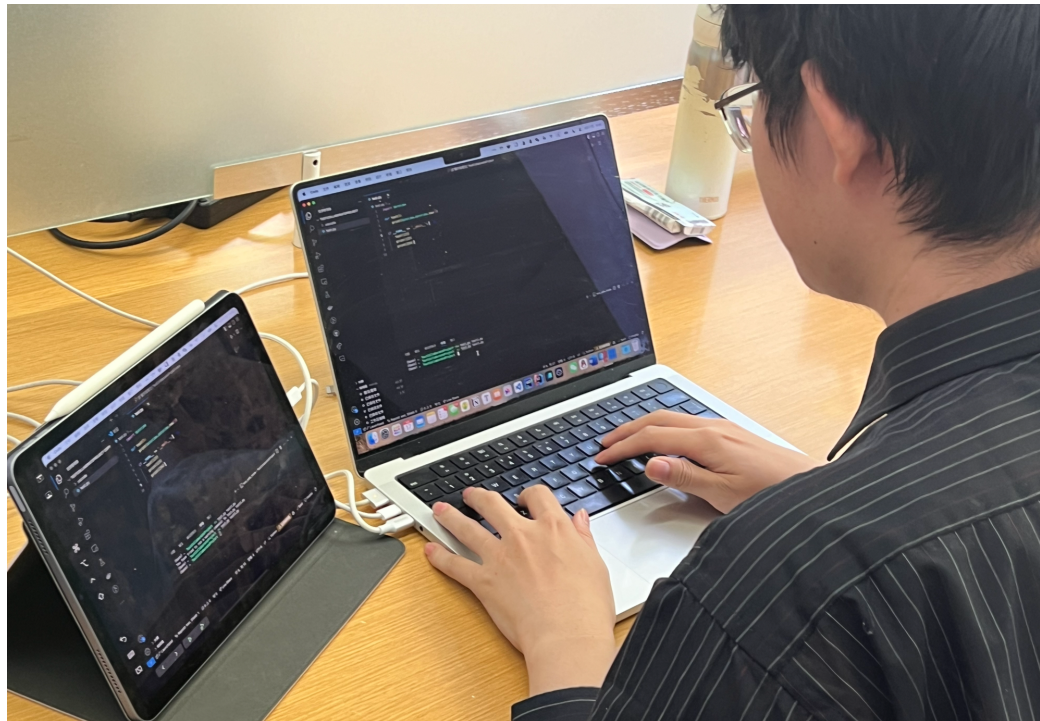


图 5.24 用户正在体验 CoVsCode 及 FlexSecure 共享终端功能

本系统的 SUS 平均得分为 82.8，表明参与者认为本系统具有良好的可用性。然而，参与者也提出了一些改进建议，如：

（1）FlexSecure 共享终端的输入延迟较大：在协作过程中，部分参与者反映输入延迟问题影响了协同效率。这种延迟在高频互动时尤为明显。

（2）协作成员不明确：在使用共享终端时，参与者无法明确知道有哪些其他成员正在同时协作，导致有时无法高效分配任务和管理工作进度。

这些反馈对系统的后续优化具有重要意义，为未来的进一步改进工作提供了灵感和方向。未来，我们计划通过优化网络传输和提升系统响应速度来减少输入延迟，同时增加协作成员可视化功能，以增强用户体验和协作效率。通过持续优化，我们希望能够提供一个更加高效和用户友好的协同编程平台。

Usability Scale

1. Based on your recent experience with us, please indicate whether you agree or disagree with the following statements:

	Strongly Disagree 1	2	3	4	Strongly Agree 5
I think that I would like to use it frequently.	☐	☐	☐	☐	☐
I found it to be simple.	☐	☐	☐	☐	☐
I thought it was easy to use.	☐	☐	☐	☐	☐
I think that I could use it without the support of a technical person.	☐	☐	☐	☐	☐
I found the various functions in it were well integrated.	☐	☐	☐	☐	☐
I thought there was a lot of consistency in it.	☐	☐	☐	☐	☐
I would imagine that most people would learn to use it very quickly.	☐	☐	☐	☐	☐
I found it very intuitive.	☐	☐	☐	☐	☐
I felt very confident using it.	☐	☐	☐	☐	☐
I could use it without having to learn anything new.	☐	☐	☐	☐	☐

2. SUS Score

Submit

图 5.25 SUS 问卷

6 总结与展望

6.1 总结

在软件工程的领域中，实时协同编程逐渐成为一种重要的开发模式，它允许多位开发者在不同的地点同时对同一项目进行编辑和调整，促进了协作的质量水平。本论文的研究工作聚焦在此背景下，旨在提升协同编程环境中的协作能力，尤其是共享终端方面的问题，以促进开发者之间的高效合作。

本文的核心成果是提出了一种无约束的多层次权限安全共享终端方案——FlexSecure，并实现了一个适用于 VS Code 的方案原型，同时与 CoVsCode 系统相结合。在研究的初期，对现有实时协同编程工具的共享终端能力进行了深入分析，明确了其主要的不足之处，尤其是在安全性、平等性等方面的局限性。基于这些认识，本设计明确了系统应具备的功能性需求和非功能性需求，实现了实时协作编程中的共享终端，并针对这些需求提出了相应的解决策略。FlexSecure 与目前市面上主流的 Live Share 与 Code With Me 的比较如表 6.1 所示。

表 6.1 FlexSecure 与其他共享终端工具的比较

功能	VS Code Live Share	JetBrains Code With Me	FlexSecure
共享终端协作	支持	支持	支持
主持人发起共享终端	支持	支持	支持
参与者发起共享终端	不支持	不支持	支持
禁止风险指令	不支持	不支持	支持

整个原型系统的实现严格按照设计的通用系统架构进行，经过了系统性的测试验证，确保了正确性和性能的要求。FlexSecure 终端的实用性和有效性通过一系列实际操作演示得到了证明，用户体验得到了提升。

总结来看，本文分析、设计并实现了一个创新的 FlexSecure 共享终端方案，并且在 CoVsCode 的基础上进行了插件的原型实现，展示了其实际应用价值。该研究工作填补了现有实时协同编程工具在共享终端方面的缺陷，提供了一种新的可能性，为实时协同编程的未来发展提供了宝贵的经验和参考。

6.2 展望

总结来看，本文提出并实现了一个更安全，更灵活的共享终端插件，提升了协作的效率。但是，由于时间和资源等限制，本系统仍然存在一些优化的地方，主要包括以下几个方面。

1. 更灵活的权限分配

细化每位成员的权限，例如赋予一部分成员只读权限，赋予另一部分成员安全权限，从而让权限分配更加灵活。这不仅能够更好地满足不同场景下的需求，还可以提高团队的工作效率和协

作体验。

2. 更好的协作感知

在共享终端中显示当前有哪些用户正在该终端中进行操作，有助于提升协作效率。通过这种方式，团队成员可以实时了解彼此的操作动态，避免冲突操作，提高整体协作的顺畅度和效率。

3. 更安全的连接方式

尽管目前方案已经采取了一定的安全措施，但随着网络安全威胁的不断演变，我们仍需持续关注并加强安全防护措施。例如，可以在共享终端时基于 CoVsCode 的身份进行认证，以避免不法分子入侵。

4. 更全面的命令拦截

目前的方案主要是在终端的输入层面进行了安全检测，然而，有可能存在不法分子通过脚本入侵电脑的情况，对此，我们需要对输入做更加全面详细的判断，以避免隐含的危险指令没有被发现，对发起者的电脑造成不可逆的损伤。

参考文献

- [1] Whitehead J. Collaboration in software engineering: A roadmap[C]. Future of Software Engineering (FOSE'07). IEEE, 2007: 214-225.
- [2] Alistair Cockburn, Laurie Williams. The costs and benefits of pair programming[M]//In Extreme Programming Examined, Addison-Wesley: United States, 2001: 223.
- [3] Saulius Astromskis, Gabriele Bavota, Andrea Janes, Barbara Russo, and Massimiliano Di Penta. 2017. Patterns of developers behaviour: A 1000-hour industrial study. Journal of Systems and Software 132 (2017), 85-97.
- [4] Spinellis D. Version control systems[J]. IEEE software, 2005, 22(5): 108-109.
- [5] Brindescu C, Codoban M, Shmarkatiuk S, et al. How do centralized and distributed version control systems impact software changes?[C]//Proceedings of the 36th international conference on Software Engineering. 2014: 322-333.
- [6] Spinellis D. Git[J]. IEEE software, 2012, 29(3): 100-101.
- [7] Zhang J. An investigation of technology design features for supporting real-time collaborative programming in an educational environment[J]. 2018.
- [8] Parra E, Alahmadi M, Ellis A, et al. A comparative study and analysis of developer communications on Slack and Gitter[J]. Empirical Software Engineering, 2022, 27(2): 40.
- [9] Williams L, Kessler R R, Cunningham W, et al. Strengthening the case for pair programming[J]. IEEE software, 2000, 17(4): 19-25.
- [10] Nosek J T. The case for collaborative programming[J]. Communications of the ACM, 1998, 41(3): 105-108.
- [11] Visual Studio Marketplace. 2023. Microsoft Visual Studio Live Share. Retrieved Nov, 2023 from <https://marketplace.visualstudio.com/items?itemName=MS-vsliveshare.vsliveshare>
- [12] Teletype for Atom. URL: <https://teletype.atom.io/#getting-started>.
- [13] JetBrains Marketplace. 2023. Code With Me. Retrieved Nov, 2023 from <https://plugins.jetbrains.com/plugin/14896-code-with-me>
- [14] Kurniawan A, Soesanto C, Wijaya J E C. Coder: Real-time code editor application for collaborative programming[J]. Procedia Computer Science, 2015, 59: 510-519.
- [15] Steinmacher I, Chaves A P, Gerosa M A. Awareness support in distributed software development: A systematic review and mapping of the literature[J]. Computer Supported Cooperative Work (CSCW), 2013, 22: 113-158.
- [16] Cho, B., Sun, C., & Ng, A. (2019). Issues and experiences in building heterogeneous co-editing systems. Proceedings of the ACM on Human-Computer Interaction, 3(GROUP). <https://doi.org/10.1145/3361126>.
- [17] Rantala M R J. Real-time collaborative coding-technical and group work challenges[D]. , 2015.
- [18] Fan, H., Sun, C., & Shen, H. (2012). ATCoPE: Any-time collaborative programming environment

for seamless integration of real-time and non-real-time teamwork in software development. GROUP'12 - Proceedings of the ACM 2012 International Conference on Support Group Work, 107–116. <https://doi.org/10.1145/2389176.2389194>.

[19] Ma, Y., Yang, Z., Chiu, B., Zhang, Y., Jiang, J., Du, B., & Fan, H. (2021). Supporting Cross-Platform Real-Time Collaborative Programming: Architecture, Techniques, and Prototype System. In International Conference on Collaborative Computing: Networking, Applications and Worksharing (pp. 124-143). Springer, Cham.

[20] Fan, H., & Sun, C. (2012). Achieving integrated consistency maintenance and awareness in real-time collaborative programming environments: The CoEclipse approach. Proceedings of the 2012 IEEE 16th International Conference on Computer Supported Cooperative Work in Design, CSCWD 2012, 94–101. <https://doi.org/10.1109/CSCWD.2012.6221803>.

[21] Lacave C, García M A, Molina A I, et al. COLLECE-2.0: A real-time collaborative programming system on Eclipse[C]//2019 International Symposium on Computers in Education (SIIE). IEEE, 2019: 1-6.

[22] Feldman M S. CodeSync: A collaborative coding environment for novice Web developers[J]. Wellesley College: Wellesley, MA, USA, 2014.

[23] Agustina, Liu F, Xia S, et al. CoMaya: incorporating advanced collaboration capabilities into 3d digital media design tools[C]//Proceedings of the 2008 ACM conference on Computer supported cooperative work. 2008: 5-8.

[24] Goldman, M., Little, G., & Miller, R. C. (2011). Real-time collaborative coding in a web IDE. UIST'11 - Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology, 155–164. <https://doi.org/10.1145/2047196.2047215>

[25] Lautamäki J, Nieminen A, Koskinen J, et al. CoRED: browser-based Collaborative Real-time Editor for Java web applications[C]//Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work. 2012: 1307-1316.

[26] Ma Y, Qi B, Xu W, et al. Integrating Real-Time and Non-Real-Time Collaborative Programming: Workflow, Techniques, and Prototypes[J]. Proceedings of the ACM on Human-Computer Interaction, 2023, 7(GROUP): 1-19.

[27] Tim Anderson. 2021. A swarm in May is worth a load of hay, is it? JetBrains Code With Me collaborative programming tool released. Retrieved Dec, 2022 from https://www.theregister.com/2021/04/07/jetbrains_code_with_me/

[28] Visual Studio Code Extension API. Available online: <https://code.visualstudio.com/api> (accessed on 1 June 2024).

[29] Hongfei Fan et al. CoVSCode: A Novel Real-Time Collaborative Programming Environment for Lightweight IDE[J]. Applied Sciences, 2019, 9(21)

[30] Tan X, Lv X, Jiang J, et al. Understanding Real-time Collaborative Programming: a Study of Visual Studio Live Share[J]. ACM Transactions on Software Engineering and Methodology, 2024.

[31] Bangor A, Kortum P T, Miller J T. An empirical evaluation of the system usability scale[J]. Intl. Journal of Human-Computer Interaction, 2008, 24(6): 574-594.

装
订
线

谢 辞

过去十八岁没戴表，不过有时间。够我没有后顾，野性贪玩。

大学四年的本科时光转瞬即逝。刚入学时，我还是懵懂的做题家；而如今，我已经能初步形成自己的世界观和价值观，确实能感受到自己的成长与进步。前三年的生活与疫情紧密相关，也算经历了一些风雨。初步实现独立，购入了苹果全家桶，购入了显示器，购入了相机和镜头，以前不敢想象的事情，随着长大都变为了现实。我也到处旅游，开阔眼界，尝试了许多以前不敢做的事。写完最后一段毕业论文，我的本科生活也随之画上句号，而接下来是新的阶段，新的开始。一路走来，只觉得自己是何其幸运。

感谢我的导师范鸿飞，在实时协同编程的研究方向上给予我很多建议和指导，在我准备保研期间也给了我很多的帮助。范老师自身知识渊博，为人友善，对待学生从不居高临下，为人处事也为学生着想，为人师表本应如此。

感谢汪明杰学长和蒋金峰学长，在课程学习方面给了我很多帮助，在论文方面给予了我很多指导，也对我的个人成长和未来规划提出了很多建议。

感谢我的女朋友沈芷怡。相处四年，我们彼此已再熟悉不过。大半个上海已留下我们的足迹，而青春的故事里，也已经写上你的名字。爱是付出，爱是给予，生命原本并没有意义，但所幸你让这个悲剧的命题能得到缓解。我对感情总是认真负责。希望我们未来也能继续走下去。纵然有风雨，我们亦携手前行。

感谢我的室友彭浩，范珑骁，王林飞。大学搬了两次宿舍，但我们仍然有幸能成为四年室友。谈笑有鸿儒，往来无白丁，特别是在寝室里共同抗疫的经历更是终身难忘。

也要感谢许许多多的朋友们，包括学姐梁潇怡，学长安江涛，王立友给予了我很多的指导，同学刘畅和我交流了很多技术心得，同学杨奕晨，胡骏宇一直和我保持着联系，同学朱怡潇，周依婷，颜浚宇也在我困难的时候给了我帮助。感谢 Johnny, Alisa 等 SAP 的同事，让我获得了宝贵的实习经历。感谢 CSDIY，让我能接触到不少优秀的课程。辅导员陈璞皎，班主任葛蕾，教务处王彩霞，本科生导师沈莹等老师也给了我很多的帮助。软件学院的各位老师也让我在软件工程领域学到了不少知识。

感谢我的父母等亲人，他们可能不懂我做的内容，但是他们依然坚定地相信我的选择，支持我的选择，肯定我的努力，并给我提供物质和精神上的支持。

知易行难，这些年读了不少书，看了不少课程，然而落实起来却是百般艰难。希望自己可以永远相信自己的主观能动性，能明白心之所向，并愿意从此找到通往当下的途径。希望自己能为生活疲惫，多做重要不紧急象限里的事情。多开阔眼界，去感受世界。希望自己懂得与人相处，不失礼貌，但也不失去自己的纯真。希望自己学会课题分离，不逃避自己的责任，也不给自己太大的压力。希望自己保持输入知识，保持锻炼身体，灵魂与肉体皆强劲。也希望自己能多赚钱，多给自己底气去感受生活，追逐自己的目标。

时间尚早，别张开眼睛。